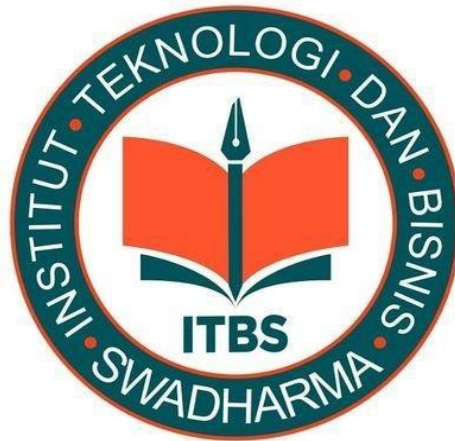


**PERANCANGAN DAN IMPLEMENTASI ALGORITMA
SUPPORT VECTOR MACHINE (SVM) DENGAN TEKNIK
STRING NORMALIZATION UNTUK DETEKSI KOMENTAR
SPAM JUDI ONLINE PADA YOUTUBE BERBASIS CHROME
EXTENSION**

**Diajukan Sebagai Salah Satu Syarat Untuk Memperoleh
Gelar Sarjana Strata Satu (S1)**

**FAKULTAS TEKNOLOGI
PROGRAM STUDI TEKNIK INFORMATIKA**



Disusun Oleh :

Nama : Yusuf Wandana

NIM : 221232017

**INSTITUT TEKNOLOGI DAN BISNIS
SWADHARMA
JAKARTA
2026**



**ITB SWADHARMA
FAKULTAS TEKNOLOGI
PROGRAM STUDI TEKNIK INFORMATIKA**

LEMBAR PERSETUJUAN DOSEN PEMBIMBING

Telah diperiksa dan disetujui sebuah skripsi untuk diajukan pada sidang skripsi, atas nama:

Nama : Yusuf Wandana
NIM : 221232017
Program Studi : Teknik Informatika
Jenjang Pendidikan : Strata Satu (S1)
Judul Skripsi : Perancangan dan Implementasi Algoritma Support Vector Machine (SVM) dengan Teknik String Normalization untuk Deteksi Komentar Spam Judi Online Pada YouTube Berbasis Chrome Extension

Jakarta,

Dosen Pembimbing

(Dr. Usanto S., S.Kom., M.Kom)

Pembimbing I

(Septiana Ningtyas, S.Kom., M.Kom)

Pembimbing II



**ITB SWADHARMA
FAKULTAS TEKNOLOGI
PROGRAM STUDI TEKNIK INFORMATIKA**

LEMBAR PENGESAHAN SKRIPSI

Pada hari tanggal, telah dipresentasikan Skripsi dihadapan para penguji untuk diuji/disidangkan atas nama :

Nama : Yusuf Wandana
NIM : 221232017
Program Studi : Teknik Informatika
Jenjang Pendidikan : Strata Satu (S1)
Judul Skripsi : Perancangan dan Implementasi Algoritma Support Vector Machine (SVM) dengan Teknik String Normalization untuk Deteksi Komentar Spam Judi Online Pada YouTube Berbasis Chrome Extension

Dan dinyatakan **Lulus** oleh para penguji.

Jakarta,

Penguji

(.....)

Penguji 1

(.....)

Penguji 2

(.....)

Penguji 3

(.....)

Penguji 4

Program Studi Teknik Informatika

(Dr. Adi Sopian, S.Kom., M.Kom)

Ketua Program Studi



LEMBAR PERNYATAAN KEASLIAN SKRIPSI

Yang bertanda tangan di bawah ini, Saya:

Nama : Yusuf Wandana
NIM : 221232017
Program Studi : Teknik Informatika
Jenjang Pendidikan : Strata Satu (S1)
Judul Skripsi : Perancangan dan Implementasi Algoritma Support Vector Machine (SVM) dengan Teknik String Normalization untuk Deteksi Komentar Spam Judi Online Pada YouTube Berbasis Chrome Extension

Menyatakan sesungguhnya:

Skripsi yang dibuat dengan judul diatas merupakan karya asli (bukan hasil plagiat) dari Saya. Bagian-bagian tertentu dalam penulisan skripsi ini yang saya kutip dari hasil karya orang lain telah dituliskan sumbernya secara jelas sesuai norma, kaidah, dan etika penulisan ilmiah. Apabila dikemudian hari ada pihak yang berkeberatan karena karya tulisnya mirip / sama dengan skripsi saya, atau ada pihak yang mengetahui bahwa skripsi Saya mirip dengan karya tulis orang lain dan orang itu menggugat kepada Saya, Saya bersedia bertanggung jawab untuk melindungi hasil tulisan Saya, serta jika ternyata benar bahwa hasil tulisan saya adalah hasil dari plagiat, maka Saya bersedia ITB Swadharma membatalkan gelar akademik yang Saya peroleh.

Demikian Lembar Pernyataan ini Saya buat dengan sebenar-benarnya dan tanpa paksaan dari pihak manapun.

Jakarta,

Yang memberi pernyataan

Yusuf Wandana

LEMBAR PERSEMBAHAN DAN MOTTO

Persembahan:

Moto:

KATA PENGANTAR

Puji dan syukur penulis panjatkan ke hadirat Tuhan Yang Maha Esa, karena atas rahmat dan karunia-Nya, skripsi yang berjudul “Perancangan dan Implementasi Algoritma Support Vector Machine (SVM) dengan Teknik String Normalization untuk Deteksi Komentar Spam Judi Online Pada Youtube Berbasis Chrome Extension” dapat diselesaikan dengan baik.

Meskipun penyusunan ini telah selesai, tetap disadari skripsi ini masih memiliki banyak aspek yang harus lebih dikembangkan lagi. Baik itu dari sisi pemilihan teknologi yang digunakan, performa yang seharusnya bisa lebih dioptimalkan, hingga pemilihan kata dalam penulisan yang juga bisa ditingkatkan. Oleh karena itu, sangat diharapkan adanya pengembangan serta penelitian lebih lanjut yang dapat memperbaiki kekurangan dari sistem saat ini.

Akhir kata, perkenankanlah untuk menyampaikan rasa hormat dan terima kasih kepada pihak-pihak yang telah membantu di dalam penyusunan skripsi ini, baik secara langsung maupun tidak langsung. Khususnya kepada:

1. Bapak Dr. Usanto S., S.Kom., M.Kom., selaku Dekan Fakultas Teknologi
2. Bapak Dr. Adi Sopian, S.Kom., M.Kom selaku Ketua Program Studi Teknik Informatika
3. Bapak Dr. Usanto S., S.Kom., M.Kom., selaku Dosen Pembimbing 1
4. Ibu Septiani Ningtyas, S.Kom., M.Kom., selaku Dosen Pembimbing 2
5. Seluruh staff ITB Swadharma yang tidak bisa disebut satu per satu
6. Seluruh keluarga yang selalu mendukung disegala keadaan
7. Serta teman-teman yang senantiasa memberikan energi positif

Akhir kata dengan kerendahan hati, semoga skripsi ini dapat memberikan dampak dan inspirasi yang positif, serta dapat dijadikan referensi untuk penelitian-penelitian di masa yang akan datang. Semoga di penelitian selanjutnya, sistem kecil ini dapat berkembang jauh lebih baik lagi dari segala aspek.

Jakarta, 2026

Penulis

DAFTAR ISI

LEMBAR PERSETUJUAN DOSEN PEMBIMBING	ii
LEMBAR PENGESAHAN SKRIPSI	iii
LEMBAR PERNYATAAN KEASLIAN SKRIPSI	iv
LEMBAR PERSEMBAHAN DAN MOTTO	v
KATA PENGANTAR	vi
DAFTAR ISI	viii
DAFTAR GAMBAR	xi
DAFTAR TABEL	xii
ABSTRAK	xiii
BAB I PENDAHULUAN	1
1.1 Latar Belakang	1
1.2 Rumusan Masalah	3
1.3 Tujuan dan Kegunaan Hasil Penelitian	4
1.3.1 Tujuan Penelitian	4
1.3.2 Kegunaan Penelitian	4
1.4 Metode Penelitian	6
1.4.1 Studi Pustaka (Literature Review)	6
1.4.2 Observasi dan Ekstraksi Data (Data Scraping)	7
BAB II Landasan Teori	Error! Bookmark not defined.
2.1 Pemrosesan Bahasa Alami (Natural Language Processing)	9
2.2 Prapemrosesan Teks (<i>Text Preprocessing</i>)	10
2.3 <i>Stemming</i> Bahasa Indonesia (Sastrawi)	11
2.4 String Normalization dan Penanganan Obfuscation	11
2.5 Ekstraksi Fitur Term Frequency-Inverse Document Frequency	12
2.6 Algoritma Support Vector Machine (SVM)	17
2.6.1 Evaluasi Model Klasifikasi	19
2.7 Algoritma Pembandingan	21
2.7.1 Multinomial Naive Bayes	21
2.7.2 Logistic Regression	22
2.8 N-gram	22

2.9 Metode Pengembangan Perangkat Lunak (<i>Waterfall</i>)	23
2.10 Perbandingan Metode Machine Learning dan Deep Learning	25
2.11 Web Scraping dan Analisis Heuristik	26
2.12 Spam Komentar pada Media Sosial (YouTube)	27
2.13 Arsitektur Chrome Extension (Manifest V3)	28
2.14 REST API dan Lingkungan FastAPI (Python)	28
2.15 Unified Modeling Language (UML)	29
2.15.1 Use Case Diagram	30
2.15.2 Activity Diagram	30
2.15.3 Sequence Diagram	31
2.16 Metode Analisis Sistem (PIECES)	32
2.17 Studi Penelitian Terdahulu	34
2.18 Persyaratan Sistem Konseptual	36
BAB III METODE PENELITIAN	Error! Bookmark not defined.
3.1 Tinjauan Umum Objek Penelitian	40
3.1.1 <i>Behaviour</i> (Tingkah Laku) Objek Penelitian	40
3.1.2 Struktur/Kategori Objek Penelitian	42
3.2 Sistem Berjalan	43
3.2.1 Tinjauan Umum	43
3.2.2 Analisis Proses Sistem Berjalan	43
3.2.3 Flowchart Sistem Berjalan	45
3.3 Permasalahan Sistem (PIECES)	46
BAB IV ANALISA PERANCANGAN SISTEM	51
4.1 Analisis Kebutuhan Sistem	51
4.1.1 Kebutuhan Fungsional	51
4.1.2 Kebutuhan Non-Fungsional	52
4.1.3 Kebutuhan Perangkat Lunak	53
4.1.4 Kebutuhan Perangkat Keras	54
4.2 Metode Pengembangan Sistem	54
4.3 Rancangan Dataset	56
4.3.1 Sumber dan Metode Pengumpulan Data	56
4.3.2 Proses Pelabelan Data	57

4.3.3 Komposisi dan Pembagian Dataset.....	58
4.4 Pipeline Sistem.....	59
4.4.1 Alur Pelatihan Model (<i>Training Pipeline</i>)	59
4.4.2 Alur Inferensi Real-Time (<i>Inference Pipeline</i>)	62
4.5 Perancangan Model.....	64
4.5.1 Konfigurasi Ekstraksi Fitur TF-IDF.....	64
4.5.2 Konfigurasi Model Linear SVM	65
4.5.3 Pencarian Nilai Parameter Terbaik (<i>GridSearchCV</i>)	65
4.5.4 Validasi Model (<i>5-Fold Cross-Validation</i>).....	66
4.5.5 Pertimbangan <i>Stemming</i>	66
4.5.6 Pertimbangan Pendekatan Hibrida.....	67
4.6 Arsitektur Komponen Sistem	67
4.6.1 Komponen <i>Scraper</i> (Node.js)	68
4.6.2 Komponen <i>Chrome Extension</i> (Manifest V3).....	68
4.6.3 Komponen Server FastAPI (Python)	69
4.7 Diagram <i>Use Case</i>	70
4.8 Activity Diagram.....	73
4.9 Sequence Diagram	75
4.10 Rancangan Tampilan.....	75
4.11 Pengujian Sistem.....	75
4.12 Analisis Kelayakan Sistem.....	75

DAFTAR PUSTAKA

DAFTAR GAMBAR

Gambar 2.1 Contoh Matriks Bobot TF-IDF pada Komentar YouTube.....	16
Gambar 2.2 Ilustrasi Hyperplane dan Margin pada Algoritma Linear SVM.....	18
Gambar 2.3 Tahapan Metode Waterfall	24
Gambar 2.4 Simbol-Simbol Use Case Diagram	30
Gambar 2.5 Simbol-Simbol Activity Diagram	31
Gambar 2.6 Simbol-Simbol Sequence Diagram	32
Gambar 3.1 Flowchart Sistem Berjalan	45
Gambar 4.1 Alur Pelatihan Model (Training Pipeline).....	61
Gambar 4.2 Alur Inferensi Real-Time (Inference Pipeline).....	63
Gambar 4.3 Diagram Arsitektur Komponen Sistem	68
Gambar 4.4 Use Case Diagram Sistem Deteksi Spam Komentar YouTube	71
Gambar 4.5 Activity Diagram Sistem Deteksi Spam Komentar YouTube	73

DAFTAR TABEL

Tabel 2.1 Aspek-Aspek Analisis PIECES	33
Tabel 3.1 Ringkasan Analisis PIECES Sistem Berjalan	48
Tabel 4.1 Kebutuhan Perangkat Lunak	53
Tabel 4.2 Kebutuhan Perangkat Keras	54
Tabel 4.3 Komposisi Dataset.....	58

ABSTRAK

BAB I

PENDAHULUAN

1.1 Latar Belakang

Perkembangan pesat teknologi informasi telah mengubah pola komunikasi dan distribusi informasi secara fundamental. Media sosial dan platform berbagi video, khususnya YouTube, kini tidak lagi sekadar menjadi sarana hiburan, melainkan telah berevolusi menjadi ruang publik digital yang memudahkan interaksi jutaan pengguna setiap harinya. Uraian interaksi tersebut umumnya terwujud dalam kolom komentar, di mana pengguna dapat saling bertukar pandangan secara bebas. Sayangnya, tingginya tingkat interaksi partisipatif ini membawa dampak negatif berupa eksploitasi celah keamanan informasi oleh pihak-pihak yang tidak bertanggung jawab.

Salah satu bentuk eksploitasi yang saat ini menjadi krisis siber di Indonesia adalah penyebaran promosi judi *online* secara ilegal (Herawati dkk., 2025). Penyebaran ini tidak dilakukan secara manual, melainkan dikoordinasikan menggunakan jaringan *bot* yang mampu mengirimkan ribuan komentar spam secara instan ke berbagai saluran YouTube yang memiliki tingkat kunjungan tinggi. Promosi masif ini tidak hanya merusak kenyamanan pengguna dalam berinteraksi, tetapi juga berpotensi besar menjerumuskan masyarakat ke dalam kejahatan finansial dan kecanduan perjudian.

Penyedia layanan platform sebenarnya telah mengimplementasikan sistem filtrasi berbasis kata kunci (*keyword-based filtering*) untuk menanggulangi konten bermuatan spam dan ilegal. Namun, sistem konvensional tersebut terbukti memiliki tingkat kerentanan yang tinggi. Khususnya di Indonesia, para penyebar spam

memiliki pola linguistik yang sangat dinamis, sering kali memadukan kata-kata tidak baku (slang/alay) dengan teknik penyamaran teks tingkat lanjut atau *obfuscation*. Teknik ini dilakukan dengan memanipulasi karakter (*homoglyphs*), seperti penggunaan font Unicode khusus (contoh: ***SL0T***, ***\$L0T***), penyisipan tanda baca di tengah kata (contoh: J.u.d.i), hingga substitusi huruf dengan angka (contoh: 5L0T). Akibatnya, kombinasi karakter tersebut tidak terbaca sebagai kata terlarang oleh mesin, sehingga lolos dari sistem penyaringan otomatis. (Angelo dkk., 2025).

Penelitian sejenis sebelumnya telah banyak mengeksplorasi penggunaan *Machine Learning* untuk mendeteksi spam pada pesan elektronik maupun media sosial X/Twitter (Chua et al., 2024). Namun, mayoritas penelitian tersebut mengasumsikan bahwa data teks yang diproses berada dalam format standar. Ketika dihadapkan pada teks dengan tingkat *obfuscation* tinggi atau tidak baku seperti pada komentar YouTube saat ini, akurasi model klasifikasi konvensional akan menurun drastis. Oleh karena itu, dibutuhkan tahap prapemrosesan teks yang kuat agar sistem tidak menganggap karakter manipulatif tersebut sebagai kosakata baru yang tidak memiliki makna (Khairunnisa & Adiwijaya, 2021).

Untuk menjembatani celah kelemahan tersebut, diperlukan sebuah sistem cerdas komprehensif yang memadukan tahap prapemrosesan teks yang kuat dengan algoritma klasifikasi yang handal. Algoritma *Support Vector Machine* (SVM) dipilih karena merupakan sistem pembelajaran yang berupaya membentuk fungsi-fungsi linear dalam ruang fitur berdimensi tinggi. Algoritma ini memisahkan data dengan mencari pembatas terbaik melalui nilai jarak (*margin*) terbesar antar kelas. Agar kinerja SVM dapat maksimal, algoritma ini harus dikombinasikan dengan teknik prapemrosesan pada tahap awal untuk membersihkan data teks dari faktor

yang tidak diperlukan (Nanda et al., 2022). Dalam konteks penelitian ini, prapemrosesan difokuskan pada *String Normalization* yang berfungsi untuk membersihkan dan menormalkan kembali karakter-karakter *Unicode* manipulatif menjadi format teks ASCII standar sebelum proses klasifikasi dilakukan.

Sejalan dengan orientasi pengembangan sistem yang menuntut adanya luaran produk yang fungsional, integrasi model deteksi ini akan diimplementasikan ke dalam sebuah *Chrome Extension*. Ekstensi *web* dapat berjalan di latar belakang peramban dan memiliki kemampuan untuk memanipulasi elemen-elemen pada halaman *web* yang sedang ditampilkan secara langsung, termasuk memodifikasi atau mengganti teks. Pemilihan pendekatan berbasis klien (*client-side*) ini bertujuan agar penapisan komentar spam dapat dieksekusi dengan memanfaatkan manipulasi *Document Object Model* (DOM) yang diinjeksi pada halaman *web* (Ramadhan & Fauzan, 2023). Dengan demikian, ekstensi dapat menyembunyikan komentar terlarang secara mandiri tanpa harus mengubah struktur pangkalan data (*database*) utama milik YouTube.

1.2 Rumusan Masalah

Berdasarkan identifikasi masalah pada latar belakang yang telah diuraikan, maka perumusan masalah dalam penelitian ini adalah: Bagaimana merancang dan mengimplementasikan algoritma *Support Vector Machine* (SVM) yang dikombinasikan dengan teknik *String Normalization* ke dalam sebuah ekstensi peramban Google Chrome berbasis arsitektur *Manifest V3*, guna mendeteksi dan menyembunyikan komentar spam promosi judi online berbahasa Indonesia yang menggunakan teknik penyamaran teks (*text obfuscation*) di platform YouTube secara *real-time*?

1.3 Tujuan dan Kegunaan Hasil Penelitian

1.3.1 Tujuan Penelitian

Sesuai dengan rumusan masalah yang telah ditetapkan, tujuan utama dari penelitian ini adalah membangun sebuah sistem filtrasi cerdas berbasis algoritma *Support Vector Machine* (SVM) dan teknik *String Normalization* guna mendeteksi serta menyembunyikan komentar spam promosi judi online di platform YouTube. Secara lebih terperinci, penjabaran dari tujuan utama tersebut meliputi:

1. Merancang dan mengimplementasikan perangkat lunak berupa ekstensi Google Chrome (*Chrome Extension*) menggunakan arsitektur *Manifest V3* yang mampu beroperasi di sisi klien (*client-side*) secara *real-time*, sehingga dapat memoderasi komentar tanpa membebani server utama platform.
2. Menerapkan teknik *String Normalization* sebagai tahap prapemrosesan awal yang vital untuk membersihkan dan menormalkan kembali karakter manipulatif (*obfuscation*) seperti penggunaan *homoglyphs*, penyisipan simbol, atau kombinasi alfanumerik menjadi format teks ASCII standar agar dapat dikenali oleh mesin.
3. Mengukur, menganalisis, dan memvalidasi performa algoritma klasifikasi *Support Vector Machine* (SVM) dalam membedakan komentar wajar (*ham*) dan komentar spam, menggunakan parameter pengujian yang komprehensif berdasarkan metrik akurasi (*accuracy*), presisi (*precision*), dan *recall*.

1.3.2 Kegunaan Penelitian

Hasil dari penelitian ini diharapkan tidak hanya menghasilkan luaran berupa perangkat lunak yang fungsional, tetapi juga memberikan dampak yang bermanfaat dan terukur. Kegunaan penelitian ini dibagi menjadi dua aspek utama, yaitu:

1. Kegunaan Teoritis (Akademis).

a) Pengembangan Ilmu Pengetahuan

Menjadi kontribusi dan referensi keilmuan baru di bidang penambangan teks (*text mining*) dan *Machine Learning*, khususnya yang berkaitan dengan pemrosesan bahasa alami (NLP) pada data teks berbahasa Indonesia yang bersifat dinamis dan tidak baku.

b) Uji Efektivitas Metode

Memberikan landasan empiris mengenai efektivitas penggabungan teknik *String Normalization* dengan algoritma SVM dalam menembus dan mengatasi masalah manipulasi teks atau *obfuscation* yang semakin kompleks pada ruang publik media sosial.

c) Referensi Penelitian Lanjutan

Menyediakan landasan teori, kerangka sistem, serta *dataset* yang dapat dimanfaatkan oleh sivitas akademika Institut Teknologi dan Bisnis Swadharma maupun peneliti lain untuk melakukan pengembangan lebih lanjut, misalnya sebagai bahan komparasi dengan metode *Deep Learning*.

2. Kegunaan Praktis

a) Bagi Masyarakat Umum (Pengguna YouTube)

Memberikan solusi perangkat lunak yang dapat diinstal dan digunakan secara langsung sebagai lapisan perlindungan tambahan secara mandiri. Sistem ini secara otomatis akan meningkatkan kenyamanan berinteraksi (*user experience*) dan melindungi keamanan pengguna saat menelusuri konten, agar terhindar dari paparan materi promosi perjudian ilegal yang berpotensi merugikan.

b) Bagi Pengembang Perangkat Lunak

Memberikan cetak biru (*blueprint*) mengenai tata cara mengintegrasikan model *Machine Learning* ke dalam ekosistem *browser extension* agar tetap ringan dan responsif saat menyeleksi elemen *Document Object Model* (DOM) secara dinamis.

1.4 Metode Penelitian

Metode penelitian pada dasarnya merupakan cara ilmiah untuk mendapatkan data dengan tujuan dan kegunaan tertentu yang didasarkan pada ciri-ciri keilmuan, yaitu rasional, empiris, dan sistematis (Sugiyono, 2022). Melalui pendekatan ilmiah ini, sebuah penelitian tidak dilakukan secara kebetulan, melainkan mengikuti langkah-langkah terstruktur untuk memecahkan suatu permasalahan yang spesifik. Dalam konteks ilmu komputer dan rekayasa perangkat lunak, metode penelitian berfungsi sebagai kerangka kerja yang memandu seluruh tahapan analisis, perancangan, hingga pengujian sistem agar berjalan secara terarah dan terukur.

1.4.1 Studi Pustaka (Literature Review)

Studi kepustakaan merupakan serangkaian kegiatan sistematis yang berkenaan dengan metode pengumpulan data pustaka, membaca, mencatat, serta mengolah bahan penelitian secara kritis (Sari dan Asmendri, 2020). Pada tahap ini, pengumpulan referensi teoritis dilakukan secara mendalam melalui pencarian literatur yang relevan dengan topik penelitian. Sumber literatur utama difokuskan pada kajian dari jurnal-jurnal akademik nasional, publikasi ilmiah, buku referensi, serta dokumentasi teknis resmi (*official documentation*) dari pengembang peramban.

Tujuan utama dari studi pustaka ini adalah untuk membedah dan memahami konsep dasar serta formulasi matematis dari algoritma *Support Vector Machine* (SVM) dalam konteks klasifikasi teks. Selain itu, kajian ini juga menggali efektivitas penerapan teknik *String Normalization* untuk menangani modifikasi teks (*text obfuscation*), serta mempelajari arsitektur dan kebijakan keamanan pengembangan ekstensi peramban berbasis *Manifest V3*. Seluruh data teoritis yang dihimpun menjadi landasan pijakan yang krusial dalam merancang bangun sistem agar sesuai dengan standar komputasi dan kaidah keilmuan yang berlaku.

Target informasi yang ingin didapatkan dari studi pustaka ini meliputi:

1. Formulasi dan parameter optimal dalam melatih model *Support Vector Machine* (SVM) untuk klasifikasi teks.
2. Metode *String Normalization* yang paling efisien dalam menangkal manipulasi karakter Unicode (*obfuscation*).
3. Arsitektur sistem komunikasi antara klien (ekstensi peramban) dengan peladen (*backend server*) yang menampung model *Machine Learning*.

1.4.2 Observasi dan Ekstraksi Data (Data Scraping)

Observasi merupakan teknik pengumpulan data empiris yang mengedepankan pengamatan langsung terhadap objek, perilaku, dan kejadian pada situasi tertentu untuk mendapatkan data primer yang akurat (Sugiyono, 2022).

Observasi dan ekstraksi data teks pada penelitian ini diotomatisasi menggunakan teknik *web scraping* berbasis *Node.js*. Pengambilan data difokuskan pada sejumlah video YouTube yang memiliki kepadatan interaksi tinggi dan berpotensi menjadi target penyebaran spam judi *online*. Proses ekstraksi ini dikombinasikan secara sistematis dengan metode analisis heuristik yang

memberikan pembobotan (*scoring*) awal pada setiap teks yang masuk. Pendekatan heuristik ini berfungsi untuk mengidentifikasi secara cepat indikasi spam berdasarkan kepadatan karakter manipulatif (*obfuscation*) atau *keyword* tertentu, sehingga memfasilitasi dan memvalidasi proses pengumpulan data latih secara efisien.

Hasil ekstraksi kemudian dikonversi ke dalam format CSV untuk keperluan pelabelan. Observasi berbasis perangkat lunak ini bertujuan untuk:

1. Mengidentifikasi pola *obfuscation* yang paling umum digunakan *spammer* secara empiris, seperti penggunaan karakter Unicode khusus, penyisipan tanda baca, dan substitusi huruf-angka.
2. Menentukan kanal atau video prioritas yang memiliki kepadatan komentar spam tinggi sebagai sumber pengumpulan *dataset*.
3. Memastikan representativitas *dataset* yang dikumpulkan mencerminkan kondisi nyata komentar spam judi *online* di YouTube saat ini.

BAB II

LANDASAN TEORI

2.1 Pemrosesan Bahasa Alami (Natural Language Processing)

Natural Language Processing (NLP) merupakan cabang dari kecerdasan buatan (*Artificial Intelligence*) yang secara spesifik berfokus pada interaksi antara sistem komputer dan bahasa manusia. Teknologi ini dirancang agar komputer memiliki kemampuan untuk mengolah, memahami, menginterpretasikan, hingga menganalisis makna dari data teks atau bahasa alami secara efektif (Oktavia, 2024). Melalui pendekatan NLP, pemrosesan informasi dari opini atau teks masukan berskala besar dapat diekstraksi menjadi sebuah pengetahuan dan kesimpulan yang valid untuk kebutuhan analisis lanjutan (Pratama, 2026).

Namun, tantangan terbesar dalam penerapan NLP pada data media sosial adalah karakteristik teks yang sangat dinamis dan tidak terstruktur. Komentar pada platform seperti YouTube sering kali dipenuhi dengan variasi bahasa yang tidak baku, penggunaan *slang* (bahasa gaul), singkatan, hingga manipulasi teks (*obfuscation*). Dalam kondisi ini, kualitas tahap prapemrosesan teks (*text preprocessing*) secara langsung memengaruhi tingkat akurasi klasifikasi. Tanpa adanya prapemrosesan dan normalisasi yang memadai, sistem tidak akan mampu mengenali karakter yang telah dimanipulasi oleh pengguna (Khairunnisa & Adiwijaya, 2021).

Tahapan prapemrosesan ini sangat krusial dalam mengubah data teks yang kotor menjadi representasi numerik yang valid untuk diproses oleh algoritma *Machine Learning* seperti *Support Vector Machine* (SVM). Oleh karena itu, penerapan teknik NLP yang presisi menjadi fondasi mutlak dalam menangani dataset dengan tingkat

noise yang tinggi, memastikan algoritma tidak gagal saat mengekstraksi fitur teks dari komentar yang tidak baku.

2.2 Prapemrosesan Teks (*Text Preprocessing*)

Prapemrosesan teks merupakan tahap awal yang krusial dalam pipeline *Natural Language Processing* sebelum data teks dapat diproses oleh algoritma *Machine Learning*. Tujuan utama dari tahap ini adalah mengubah teks mentah yang tidak terstruktur dan penuh *noise* menjadi representasi yang bersih dan seragam sehingga algoritma ekstraksi fitur dapat bekerja secara optimal (Khairunnisa dkk., 2021).

Tantangan prapemrosesan menjadi lebih kompleks pada data teks media sosial seperti komentar YouTube, yang sering mengandung singkatan tidak standar, bahasa gaul, dan manipulasi karakter yang disengaja untuk menghindari moderasi otomatis. Kondisi ini mengharuskan pipeline prapemrosesan yang lebih komprehensif dibandingkan teks formal.

Tahapan prapemrosesan teks umumnya meliputi *case folding* untuk menyeragamkan kapitalisasi, normalisasi Unicode untuk mengonversi karakter dekoratif atau manipulatif ke bentuk standar, konversi emoji menjadi token teks yang informatif, penghapusan karakter tidak relevan seperti URL dan simbol, serta penghapusan *stopword* untuk mereduksi kata-kata fungsional yang tidak memiliki nilai diskriminatif. Dalam konteks deteksi spam judi *online*, dua tahap tambahan juga relevan: normalisasi *leet speak* yang mengonversi penggantian huruf dengan angka seperti "s1tus" kembali menjadi "situs", dan kanonisasi nama *brand* yang menyeragamkan pola penamaan situs judi *online* menjadi satu token representatif.

Hasil akhir dari seluruh tahapan ini adalah teks yang bersih dan siap diproses pada tahap ekstraksi fitur. Penjelasan teknis mengenai implementasi setiap tahap dibahas lebih lanjut pada Bab 3.

2.3 Stemming Bahasa Indonesia (Sastrawi)

Stemming adalah proses mengubah kata berimbuhan menjadi kata dasarnya dengan menghilangkan imbuhan yang melekat, sehingga kata-kata yang secara semantis sama namun berbeda bentuk morfologisnya dapat dikenali sebagai satu entitas yang identik oleh model klasifikasi (Sinaga & Nainggolan, 2023). Bahasa Indonesia memiliki sistem afiksasi yang kompleks mencakup awalan, akhiran, sisipan, dan kombinasi keduanya, sehingga dibutuhkan algoritma *stemming* yang dirancang khusus untuk kaidah morfologi bahasa Indonesia.

Sastrawi adalah pustaka Python yang mengimplementasikan algoritma Nazief-Adriani dengan kamus kata dasar bahasa Indonesia. Algoritma ini secara iteratif mencoba melepaskan imbuhan hingga ditemukan kata dasar yang valid, dan telah terbukti mencapai akurasi rata-rata 97,73% pada dokumen teks berbahasa Indonesia (Sinaga & Nainggolan, 2023). Meskipun demikian, efektivitas *stemming* pada teks komentar media sosial yang pendek dan informal tidak selalu menghasilkan peningkatan performa yang signifikan dibandingkan teks formal. Pertimbangan empiris mengenai penerapan *stemming* dalam penelitian ini dibahas pada Bab 3.

2.4 String Normalization dan Penanganan Obfuscation

String Normalization adalah teknik krusial dalam prapemrosesan teks yang bertujuan untuk menangani anomali penulisan atau penyamaran teks (*obfuscation*). Secara lengkap, *obfuscation* merupakan teknik penyamaran kata dengan cara memodifikasi karakter, seperti penggantian huruf dengan angka atau simbol

(contohnya penggunaan *homoglyphs* atau variasi *font* Unicode khusus), yang bertujuan untuk menghindari sistem deteksi otomatis. Teknik *obfuscation* seperti substitusi karakter ini membuat sistem filtrasi berbasis kata kunci (*keyword-based filtering*) konvensional menjadi tidak efektif, khususnya dalam menangkal penyebaran promosi judi *online* di platform YouTube (Angelo dkk., 2025).

Dalam pemrosesan klasifikasi berbasis *Machine Learning*, tingkat akurasi sangat bergantung pada konsistensi kamus data (*vocabulary*). Apabila data mentah dipenuhi dengan *noise* atau karakter manipulatif, algoritma akan mengalami penurunan performa secara drastis. Jika proses prapemrosesan dan normalisasi teks tidak dilakukan secara maksimal, kinerja metode klasifikasi apa pun tidak akan cukup kuat dalam menentukan kelas kata karena kamus sistem akan menganggap karakter manipulatif tersebut sebagai kosakata baru yang tidak memiliki makna (Abdillah dkk., 2021).

Oleh karena itu, *String Normalization* berfungsi sebagai lapisan pembersih yang menyetarakan dan memetakan kembali berbagai variasi karakter Unicode manipulatif tersebut ke dalam bentuk teks ASCII standar. Melalui tahapan normalisasi ini, karakter teks yang telah diobfuskasi dikembalikan ke bentuk aslinya, sehingga mencegah kegagalan algoritma klasifikasi akibat ketidakmampuan mesin dalam membaca teks spam yang dimanipulasi.

2.5 Ekstraksi Fitur Term Frequency-Inverse Document Frequency

Ekstraksi fitur merupakan salah satu tahapan krusial dalam klasifikasi teks yang berfungsi untuk mengonversi data tekstual mentah menjadi bentuk representasi vektor numerik. Data komentar YouTube yang telah melalui tahap prapemrosesan dan normalisasi string tidak dapat diproses secara langsung oleh algoritma

komputasi berbasis *Machine Learning*, karena sistem tersebut membutuhkan masukan (*input*) berupa matriks angka. Salah satu metode ekstraksi fitur yang umum digunakan dan terbukti efektif dalam klasifikasi teks berbahasa Indonesia adalah *Term Frequency-Inverse Document Frequency* (TF-IDF) (Arrayyan dkk., 2025).

Metode TF-IDF bekerja dengan cara memberikan pembobotan statistik pada setiap kata yang terdapat di dalam suatu dokumen atau komentar. Bobot ini dihitung berdasarkan dua metrik utama, yaitu seberapa sering sebuah kata muncul di dalam satu komentar spesifik (*Term Frequency*), dan seberapa langka atau umum kata tersebut jika dibandingkan dengan keseluruhan komentar yang ada di dalam himpunan data (*Inverse Document Frequency*). Dibandingkan metode representasi teks lain seperti *Bag of Words* (BoW), Doc2Vec, dan Word2Vec, TF-IDF secara konsisten menunjukkan performa kompetitif ketika dikombinasikan dengan algoritma *machine learning* seperti SVM (Efrizoni dkk., 2022).

Secara matematis, kalkulasi nilai TF-IDF diformulasikan dalam persamaan berikut:

$$W_{t,d} = tf_{t,d} \times idf_t$$

Nilai idf_t sendiri didapatkan melalui perhitungan logaritmik dengan persamaan:

$$idf_t = \log\left(\frac{N}{df_t}\right)$$

Keterangan variabel:

1. $W_{t,d}$ (Bobot Akhir): Merupakan nilai metrik akhir dari sebuah kata (t) di dalam satu komentar tertentu (d). Semakin tinggi nilai ini, berarti kata tersebut semakin penting dan menjadi ciri khas kuat dari komentar tersebut.

2. $tf_{t,d}$ (*Term Frequency*): Merupakan jumlah atau frekuensi kemunculan kata (t) di dalam satu komentar (d). Jika kata "slot" muncul 3 kali dalam satu kalimat komentar, maka nilai (tf) untuk kata tersebut adalah 3.
3. N (Total Dokumen): Merupakan jumlah keseluruhan komentar YouTube yang ada di dalam dataset latih.
4. df_t (*Document Frequency*): Merupakan jumlah komentar (dari total N) yang mengandung kata (t). Jika ada 10.000 komentar, dan kata "slot" muncul di 500 komentar, maka nilai df adalah 500.

Sebagai ilustrasi bagaimana TF-IDF menyaring kata-kata spam perjudian, asumsikan bahwa terdapat sebuah *dataset* mini yang hanya berisi 3 komentar YouTube ($N = 3$):

1. Komentar 1 (d_1): "link slot paling gacor hari ini"
2. Komentar 2 (d_2): "video edukasi ini sangat bermanfaat"
3. Komentar 3 (d_3): "depo slot anti rungkad di sini"

Sistem akan menghitung bobot untuk kata target, misalnya kata "slot" pada Komentar 1 (d_1):

1. Hitung TF (tf): Kata "slot" muncul sebanyak 1 kali di Komentar 1, maka variabel $tf = 1$.
2. Hitung IDF (idf): Total komentar (N) adalah 3. Kata "slot" muncul di 2 komentar (1 dan 3), maka $df = 2$. Nilai $idf = \log(3/2) = \log(1.5) \approx 0.176$.
3. Hitung Bobot Akhir (W): $W = 1 \times 0.176 = 0.176$, yang artinya kata "slot" pada Komentar 1 memiliki bobot 0.176.

Sebagai perbandingan, hitung juga kata hubung umum yang sering muncul seperti "ini" pada Komentar 1 (d_1):

1. Hitung TF (tf): Kata “ini” muncul sebanyak 1 kali di Komentar 1, maka variabel $tf = 1$.
2. Hitung IDF (idf): Misalkan kata "ini" kebetulan muncul di seluruh komentar (Komentar 1, 2, dan 3), maka $df = 3$. Jadi nilai dari variabel $idf = \log(3/3) = \log(1) = 0$.
3. Hitung Bobot Akhir (W): $W = 1 \times 0 = 0$. Kata "ini" memiliki bobot 0.

Dari perbandingan di atas, terbukti bahwa TF-IDF secara otomatis memberikan bobot 0 pada kata-kata umum yang muncul di semua konteks, dan memberikan bobot tinggi pada kata-kata spesifik yang menjadi ciri khas promosi judi *online* (seperti "slot" dan "gacor"). Hasil matriks bobot inilah yang kemudian menjadi variabel masukan utama bagi algoritma klasifikasi *Support Vector Machine* (SVM) (Ardiansyah dkk., 2025).

Tabel 1.1 Contoh Matriks Bobot TF-IDF pada Komentar YouTube

A. TERM FREQUENCY (TF)

Komentar	awan4d	member	baru	jamin	gacor	berkah99	withdraw	lancar	liga	como	tonton	struktur	video	rapi	enak
K1 spam	1	1	1	1	1	0	0	0	0	0	0	0	0	0	0
K2 spam	0	0	0	0	0	1	1	1	0	0	0	0	0	0	1
K3 non-spam	0	0	0	0	0	0	0	0	1	1	1	0	0	0	1
K4 non-spam	0	0	0	0	0	0	0	0	0	0	0	1	1	1	0

B. STATISTIK IDF

Statistik	awan4d	member	baru	jamin	gacor	berkah99	withdraw	lancar	liga	como	tonton	struktur	video	rapi	enak
df	1	1	1	1	1	1	1	1	1	1	1	1	1	1	2
IDF = $\log(4/df)$	0,602	0,602	0,602	0,602	0,602	0,602	0,602	0,602	0,602	0,602	0,602	0,602	0,602	0,602	0,301

C. BOBOT AKHIR TF-IDF (W = TF × IDF)

Komentar	awan4d	member	baru	jamin	gacor	berkah99	withdraw	lancar	liga	como	tonton	struktur	video	rapi	enak
K1 spam	0,602	0,602	0,602	0,602	0,602	0	0	0	0	0	0	0	0	0	0
K2 spam	0	0	0	0	0	0,602	0,602	0,602	0	0	0	0	0	0	0,301
K3 non-spam	0	0	0	0	0	0	0	0	0,602	0,602	0,602	0	0	0	0,301
K4 non-spam	0	0	0	0	0	0	0	0	0	0	0	0,602	0,602	0,602	0

Keterangan: K1 = "di mana lagi kalo bukan di AWAN4D khusus member baru di jamin GACOR" (spam) | K2 = "Ayo bro bestie BERKAH99 ui enak lets go biar paham withdraw lancar" (spam) | K3 = "Liga Italia cuman como yang enak di tonton skrng" (non-spam) | K4 = "struktur videonya rapi, enak diikuti" (non-spam). N = 4. Kata stopword dan karakter non-ASCII telah dihapus pada tahap prapemrosesan.

0,602 = bobot tinggi (kata unik, df=1) | 0,301 = bobot lebih rendah (kata "enak" muncul di 2 dokumen, df=2) | 0 = kata tidak hadir

Sumber: diolah penulis

Gambar 2.1 Contoh Matriks Bobot TF-IDF pada Komentar YouTube

Perlu dicatat bahwa formula yang dijelaskan di atas merupakan bentuk dasar TF-IDF yang digunakan untuk membangun pemahaman konseptual. Pada implementasi sistem dalam penelitian ini, TF-IDF diterapkan menggunakan pustaka Scikit-learn dengan beberapa penyesuaian parameter. Salah satunya adalah *sublinear_tf=True*, yaitu pengaturan yang membuat perhitungan *Term Frequency* tidak dihitung secara linear berdasarkan jumlah kemunculan kata apa adanya, melainkan diubah menggunakan fungsi logaritma menjadi $1 + \log(tf)$. Tujuannya adalah untuk meredam dominasi kata yang muncul berulang kali dalam satu komentar, sehingga sebuah kata yang muncul 20 kali tidak dianggap 20 kali lebih penting dibandingkan kata yang hanya muncul sekali, melainkan hanya sedikit lebih penting. Selain itu, pustaka Scikit-learn secara bawaan (tanpa perlu diatur secara eksplisit) juga menerapkan teknik *smoothing* pada perhitungan IDF, yaitu

menambahkan angka 1 pada pembilang dan penyebut rumus IDF. Tujuannya adalah untuk mencegah kemungkinan pembagian dengan angka nol apabila terdapat kata yang tidak muncul sama sekali pada data latih. Penjelasan lebih lanjut mengenai konfigurasi parameter ini beserta justifikasi pemilihannya dibahas secara rinci pada Bab 3.

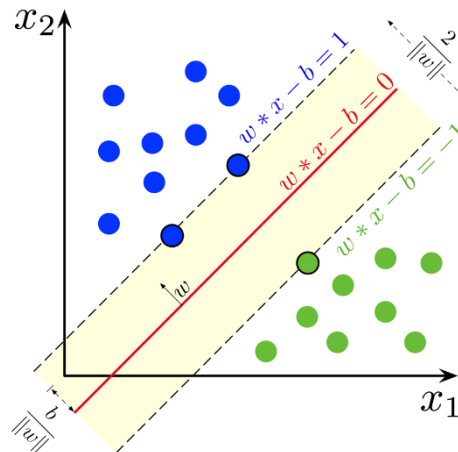
2.6 Algoritma Support Vector Machine (SVM)

Support Vector Machine (SVM) merupakan algoritma pembelajaran mesin berbasis supervised learning yang bekerja dengan cara mencari bidang pemisah terbaik (*hyperplane*) untuk membagi data ke dalam dua kelas yang berbeda. Secara matematis, *hyperplane* pemisah pada Linear SVM didefinisikan sebagai:

$$w \cdot x + b = 0$$

di mana w merupakan vektor bobot (*weight vector*) yang tegak lurus terhadap *hyperplane*, x merupakan vektor fitur dari data masukan, dan b merupakan bias. Tujuan utama SVM adalah menemukan nilai w dan b yang memaksimalkan margin, yaitu jarak antara *hyperplane* dengan titik data terdekat dari masing-masing kelas (*support vector*). Lebar margin tersebut diformulasikan sebagai:

$$\text{margin} = 2 / ||w||$$



Gambar 2.2 Ilustrasi Hyperplane dan Margin pada Algoritma Linear SVM

Memaksimalkan margin ekuivalen dengan meminimalkan $\|w\|$, sehingga permasalahan ini diselesaikan sebagai optimasi kuadratik. Algoritma ini sangat efektif dalam menangani klasifikasi teks karena kemampuannya dalam melakukan pemetaan data ke ruang dimensi yang sangat tinggi. Untuk data teks berdimensi tinggi seperti hasil ekstraksi TF-IDF, kernel linear terbukti lebih efisien dan tidak rentan terhadap *overfitting* dibandingkan kernel non-linear seperti RBF, karena data teks pada ruang TF-IDF umumnya sudah dapat dipisahkan secara linear. Penggunaan Linear SVM terbukti mampu mencapai tingkat akurasi hingga 95% dalam mendeteksi spam pada komentar YouTube, yang menunjukkan performa stabil pada data teks media sosial (Airlangga, 2024a).

Selain ketangguhannya pada data berdimensi tinggi, SVM juga dikenal andal dalam memproses teks bahasa alami yang kompleks. Penggabungan metode ekstraksi fitur seperti TF-IDF dengan algoritma SVM mampu menghasilkan akurasi di atas 96% dalam mengklasifikasikan komentar spam judi *online* berbahasa Indonesia di platform YouTube (Ardiansyah dkk., 2025). Keunggulan ini menjadikan SVM pilihan yang tepat untuk klasifikasi teks dengan variasi penulisan

tinggi, seperti komentar pada platform YouTube yang kerap menggunakan singkatan, eufemisme, dan karakter non-standar.

Berdasarkan uraian di atas, algoritma SVM merupakan metode klasifikasi yang andal untuk menangani data teks berdimensi tinggi. Kemampuannya dalam menemukan *hyperplane* optimal menjadikannya solusi yang relevan untuk diimplementasikan dalam sistem deteksi spam, terutama ketika dikombinasikan dengan teknik prapemrosesan dan ekstraksi fitur yang kuat.

2.6.1 Evaluasi Model Klasifikasi

Evaluasi model merupakan tahap yang tidak terpisahkan dari proses pengembangan sistem klasifikasi berbasis *machine learning*. Tujuan utama dari evaluasi adalah mengukur sejauh mana model yang telah dilatih mampu melakukan prediksi yang benar terhadap data yang belum pernah dilihat sebelumnya, sehingga memberikan gambaran objektif tentang performa model di dunia nyata (Helmiyah & Pramestiawan, 2025). Tanpa evaluasi yang tepat, sebuah model yang tampak baik pada data latih bisa jadi gagal total ketika dihadapkan pada data baru akibat fenomena *overfitting*.

Alat utama yang digunakan dalam evaluasi model klasifikasi biner adalah *confusion matrix*. *Confusion matrix* adalah sebuah tabel yang merangkum hasil prediksi model dengan membandingkannya terhadap label sebenarnya, menghasilkan empat komponen utama. *True Positive* (TP) adalah jumlah data yang benar-benar berlabel positif dan diprediksi benar sebagai positif. *True Negative* (TN) adalah jumlah data yang benar-benar berlabel negatif dan diprediksi benar sebagai negatif. *False Positive* (FP) adalah jumlah data negatif yang keliru diprediksi sebagai positif. *False Negative* (FN) adalah jumlah data positif yang

keliru diprediksi sebagai negatif. Dalam konteks sistem ini, kelas positif merujuk pada komentar spam dan kelas negatif merujuk pada komentar normal (*ham*).

Berdasarkan keempat komponen *confusion matrix* tersebut, terdapat empat metrik evaluasi utama yang lazim digunakan dalam klasifikasi teks. Pertama, akurasi (*accuracy*) mengukur proporsi keseluruhan prediksi yang benar terhadap total data uji, dan dirumuskan sebagai berikut:

$$Accuracy = (TP + TN) / (TP + TN + FP + FN)$$

Kedua, presisi (*precision*) mengukur seberapa tepat model ketika memberikan prediksi positif, yaitu berapa banyak dari semua komentar yang diprediksi spam yang memang benar-benar spam:

$$Precision = TP / (TP + FP)$$

Ketiga, *recall* mengukur kemampuan model dalam menemukan seluruh komentar spam yang sebenarnya ada di dalam data, yaitu berapa banyak spam yang berhasil terdeteksi dari total spam yang ada:

$$Recall = TP / (TP + FN)$$

Keempat, *F1-Score* merupakan rata-rata harmonik antara presisi dan *recall* yang memberikan keseimbangan antara keduanya. Metrik ini sangat berguna ketika distribusi kelas tidak seimbang karena tidak mudah dipengaruhi oleh kelas mayoritas:

$$F1 - Score = 2 \times (Precision \times Recall) / (Precision + Recall)$$

Dalam konteks deteksi spam komentar judi *online*, metrik *recall* mendapat perhatian khusus karena konsekuensi dari *False Negative* yakni komentar spam yang lolos dari deteksi jauh lebih merugikan pengguna dibandingkan *False Positive* yakni komentar normal yang keliru disembunyikan. Oleh karena itu, *F1-Score*

digunakan sebagai metrik evaluasi utama untuk memastikan keseimbangan antara ketepatan dan kelengkapan deteksi.

2.7 Algoritma Pemanding

Selain algoritma SVM sebagai model utama, penelitian ini juga mengimplementasikan dua algoritma pemanding dalam proses komparasi performa, yaitu *Multinomial Naive Bayes* dan *Logistic Regression*. Komparasi ini dilakukan untuk membuktikan secara empiris bahwa pemilihan SVM sebagai algoritma inti bukan sekadar asumsi, melainkan didasarkan pada hasil evaluasi kuantitatif terhadap data yang sama.

2.7.1 Multinomial Naive Bayes

Naive Bayes merupakan algoritma klasifikasi probabilistik yang bekerja berdasarkan teorema Bayes dengan asumsi independensi kondisional antar fitur. Algoritma ini menghitung probabilitas suatu dokumen teks masuk ke dalam kelas tertentu berdasarkan frekuensi kemunculan setiap kata di dalam dokumen tersebut. Varian *Multinomial Naive Bayes* secara khusus dirancang untuk data yang berbentuk frekuensi atau hitungan, sehingga sangat sesuai untuk dipadukan dengan representasi fitur TF-IDF dalam klasifikasi teks (Apricia dkk., 2024).

Keunggulan utama algoritma ini terletak pada kesederhanaan komputasinya. *Multinomial Naive Bayes* memiliki waktu pelatihan yang sangat cepat dan kebutuhan memori yang rendah karena hanya perlu menghitung probabilitas kata per kelas selama pelatihan. Karakteristik ini menjadikannya pilihan populer sebagai *baseline* dalam penelitian klasifikasi teks. Namun, asumsi independensi antar fitur yang dipegang algoritma ini dalam praktiknya jarang terpenuhi pada teks alami,

sehingga performa aktualnya sering lebih rendah dibandingkan model yang tidak membuat asumsi tersebut.

2.7.2 Logistic Regression

Logistic Regression adalah algoritma klasifikasi berbasis model linear yang menggunakan fungsi sigmoid untuk memetakan kombinasi linear dari fitur-fitur masukan ke dalam nilai probabilitas antara 0 dan 1. Dalam konteks klasifikasi biner spam, algoritma ini mempelajari bobot (*weight*) untuk setiap fitur TF-IDF sehingga dapat memperkirakan probabilitas sebuah komentar termasuk kelas spam atau *ham* (Maulana, 2025). Kelas akhir ditentukan berdasarkan nilai ambang batas (*threshold*) yang umumnya ditetapkan pada 0,5.

Dibandingkan *Naive Bayes*, *Logistic Regression* tidak membuat asumsi independensi antar fitur sehingga lebih mampu menangkap hubungan kompleks antar kata dalam teks. Algoritma ini juga relatif mudah diinterpretasikan karena bobot setiap fitur dapat diperiksa langsung untuk mengetahui kata-kata mana yang paling berpengaruh terhadap keputusan klasifikasi. Namun demikian, *Logistic Regression* cenderung kurang optimal pada data berdimensi sangat tinggi dengan batas pemisah kelas yang tidak bersifat linear, kondisi yang justru menjadi kekuatan utama SVM.

2.8 N-gram

N-gram adalah teknik ekstraksi fitur dalam pemrosesan teks yang menghasilkan urutan berkesinambungan sebanyak n token dari sebuah teks. Setiap *n-gram* merepresentasikan satu unit fitur yang dapat digunakan sebagai masukan bagi algoritma klasifikasi. Ketika n bernilai 1, teknik ini disebut *unigram* dan setiap kata diperlakukan sebagai fitur tersendiri. Ketika n bernilai 2, disebut *bigram* dan setiap

pasangan kata yang berurutan diperlakukan sebagai satu fitur. Ketika n bernilai 3, disebut *trigram* yang mencakup tiga kata berurutan sebagai satu unit fitur (Iriananda dkk., 2024).

Keunggulan utama penggunaan n -gram di atas *unigram* terletak pada kemampuannya menangkap konteks dan makna yang lebih kaya dari sebuah teks. Beberapa konsep dalam bahasa memiliki makna yang berbeda ketika berdiri sendiri dibandingkan ketika dikombinasikan, sehingga *bigram* dan *trigram* mampu merepresentasikan informasi semantik yang hilang apabila hanya menggunakan *unigram*. Sebagai contoh, frasa "slot gacor" atau "depo sekarang" dalam konteks komentar judi *online* memiliki makna yang jauh lebih spesifik sebagai satu kesatuan dibandingkan kata "slot" atau "depo" secara terpisah.

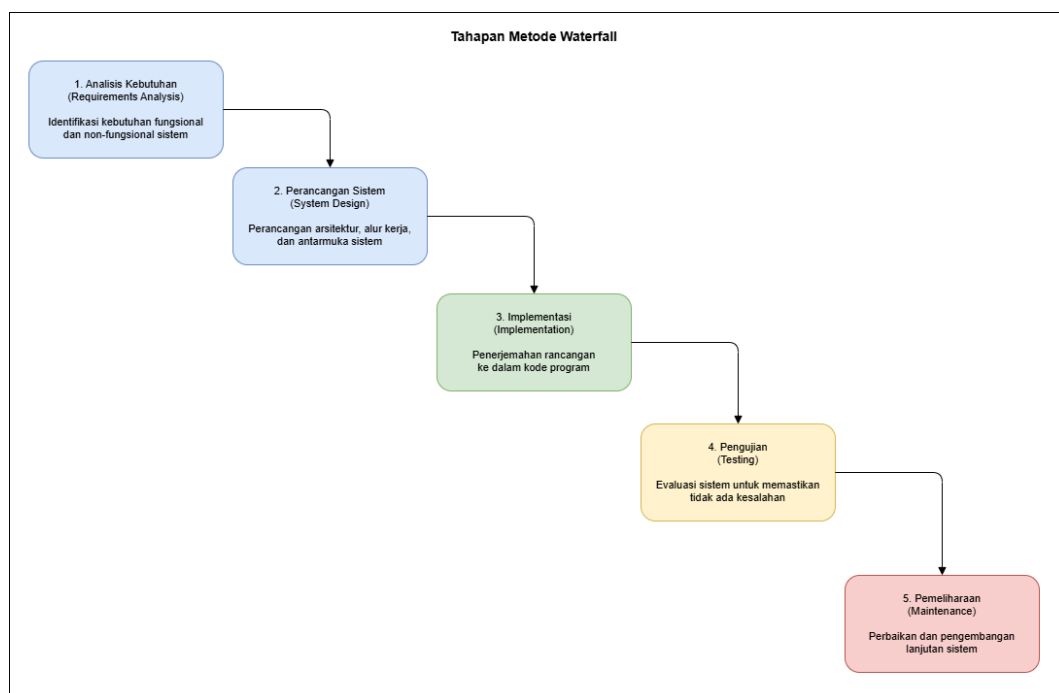
Namun, peningkatan nilai n juga membawa konsekuensi berupa pertumbuhan dimensi ruang fitur yang eksponensial. Semakin besar nilai n , semakin banyak kombinasi fitur yang dihasilkan, sehingga meningkatkan kebutuhan memori dan waktu komputasi. Selain itu, n -gram dengan nilai besar cenderung menghasilkan fitur yang sangat jarang (*sparse*) karena kombinasi kata tertentu mungkin hanya muncul sekali atau tidak sama sekali dalam dataset, yang dapat menurunkan kemampuan generalisasi model. Oleh karena itu, pemilihan nilai n yang optimal perlu dilakukan melalui eksperimen empiris untuk menemukan keseimbangan antara kekayaan representasi fitur dan efisiensi komputasi (Iriananda dkk., 2024).

2.9 Metode Pengembangan Perangkat Lunak (*Waterfall*)

Pengembangan perangkat lunak memerlukan pendekatan metodologis yang terstruktur agar proses pembangunan sistem dapat berjalan secara sistematis dan menghasilkan produk yang sesuai dengan kebutuhan yang telah ditetapkan. Salah

satu metode yang paling banyak digunakan adalah metode *Waterfall*. Metode ini bekerja secara linear dan sekuensial, di mana setiap tahap harus diselesaikan secara tuntas sebelum dapat melanjutkan ke tahap berikutnya (Anis dkk., 2024).

Metode *Waterfall* dipilih dalam situasi di mana kebutuhan sistem telah terdefinisi dengan jelas sejak awal dan tidak memerlukan perubahan spesifikasi secara berulang di tengah proses pengembangan. Karakteristik ini berbeda dengan metode *Agile* yang bersifat iteratif dan fleksibel terhadap perubahan kebutuhan. Metode *Waterfall* terdiri dari lima tahap utama yang dilaksanakan secara berurutan, yaitu analisis kebutuhan, perancangan sistem, implementasi, pengujian, dan pemeliharaan (Anis dkk., 2024).



Gambar 2.3 Tahapan Metode Waterfall

Tahap pertama adalah analisis kebutuhan (*requirements analysis*), yaitu proses identifikasi dan pendokumentasian seluruh kebutuhan fungsional dan non-fungsional sistem. Tahap kedua adalah perancangan sistem (*system design*), di mana

arsitektur sistem, alur kerja, dan antarmuka dirancang secara menyeluruh. Tahap ketiga adalah implementasi (*implementation*), yaitu penerjemahan rancangan ke dalam kode program. Tahap keempat adalah pengujian (*testing*), di mana sistem diuji untuk memastikan tidak ada kesalahan dan berjalan sesuai kebutuhan. Tahap kelima adalah pemeliharaan (*maintenance*), yaitu proses perbaikan dan pengembangan lanjutan setelah sistem digunakan.

2.10 Perbandingan Metode Machine Learning dan Deep Learning

Perkembangan metode klasifikasi teks dalam beberapa tahun terakhir menunjukkan adanya pergeseran dari pendekatan *Machine Learning* tradisional menuju metode *Deep Learning*. Algoritma *Machine Learning* seperti *Support Vector Machine* (SVM) umumnya mengandalkan fitur hasil ekstraksi manual seperti *Term Frequency-Inverse Document Frequency* (TF-IDF), yang relatif ringan secara komputasi namun tetap efektif untuk dataset berukuran kecil hingga menengah. Sebaliknya, pendekatan *Deep Learning* mampu mempelajari representasi teks secara otomatis tanpa memerlukan rekayasa fitur secara manual. Model berbasis *Deep Learning* seperti LSTM mampu mencapai tingkat akurasi tertinggi sebesar 95,65% dalam mendeteksi spam komentar YouTube, namun hal tersebut berbanding lurus dengan kebutuhan sumber daya komputasi yang lebih besar (Airlangga, 2024b).

Meskipun menawarkan akurasi yang tinggi, model *Deep Learning* menuntut kompleksitas implementasi yang lebih tinggi serta waktu pelatihan yang lebih lama dibandingkan metode tradisional. Sebaliknya, metode *Machine Learning* seperti SVM mampu memberikan performa klasifikasi yang kompetitif dengan beban

komputasi yang jauh lebih ringan, sehingga lebih praktis untuk diimplementasikan pada sistem dengan keterbatasan sumber daya (Airlangga, 2024a).

2.11 Web Scraping dan Analisis Heuristik

Dalam pengembangan kecerdasan buatan, ketersediaan himpunan data (*dataset*) yang melimpah dan relevan menjadi syarat mutlak. *Web scraping* adalah teknik mengekstrak data dalam jumlah besar secara cepat dan otomatis dari halaman web (Koprawi & Putra, 2023). Teknik ini dilakukan dengan mengirimkan permintaan protokol jaringan ke peladen situs web, kemudian memuat struktur kode HTML secara utuh, dan mem-*parsing* elemen-elemen spesifik untuk mendapatkan informasi yang dibutuhkan. Pada arsitektur sistem ini, *web scraping* berjalan secara *asynchronous* melalui *runtime environment* Node.js untuk mengekstraksi komentar-komentar pada platform YouTube tanpa mengganggu eksekusi proses lainnya.

Lebih lanjut, untuk memastikan tingkat validitas data yang ditarik, teknik *scraping* ini dikombinasikan dengan metode analisis heuristik. Dalam konteks ilmu komputer, pendekatan heuristik merupakan teknik pemecahan masalah berbasis aturan praktis (*rule-based*) yang memanfaatkan pengetahuan domain untuk menghasilkan keputusan secara efisien, meskipun tidak selalu menjamin solusi yang optimal (Kaddoura dkk., 2022). Dalam penelitian ini, heuristik diimplementasikan sebagai algoritma penyaringan (*filter*) tahap pertama yang memberikan parameter pembobotan (*scoring*) otomatis pada hasil ekstraksi. Sistem akan mendeteksi kepadatan anomali karakter (seperti manipulasi Unicode) atau kata kunci spesifik (*keywords*), sehingga proses pengumpulan data latih promosi

perjudian menjadi jauh lebih presisi dan terstruktur sebelum diteruskan ke model pelatihan SVM.

2.12 Spam Komentar pada Media Sosial (YouTube)

Spam komentar pada media sosial merupakan salah satu bentuk penyalahgunaan platform digital yang dilakukan dengan tujuan tertentu, seperti promosi, penyebaran informasi yang tidak relevan, maupun aktivitas penipuan. Pada platform berbagi video seperti YouTube, aktivitas ini umumnya dihasilkan oleh *bot* atau akun otomatis yang memposting pesan secara berulang dengan pola tertentu. *Spam* pada YouTube sering kali berasal dari sistem otomatis yang dirancang untuk menyebarkan tautan berbahaya atau promosi massal yang mengganggu kualitas interaksi pengguna (Khan, 2018).

Fenomena ini telah berkembang menjadi ancaman yang lebih serius di Indonesia, khususnya terkait penyebaran konten ilegal. Penyebaran promosi judi *online* di media sosial telah menjadi krisis siber yang mengancam ketahanan nasional, di mana para pelaku memanfaatkan kolom komentar platform besar seperti YouTube untuk menjangkau masyarakat secara masif (Herawati dkk., 2025). Hal ini menyebabkan penurunan kualitas ekosistem digital dan berpotensi menjerumuskan pengguna ke dalam risiko finansial.

Berdasarkan uraian di atas, spam pada platform YouTube telah bertransformasi dari sekadar gangguan teknis menjadi sarana penyebaran konten ilegal yang terorganisir melalui bantuan bot. Oleh karena itu, diperlukan sistem perlindungan yang mampu mendeteksi pola otomatisasi tersebut guna menjaga keamanan dan kenyamanan pengguna dalam berinteraksi di ruang publik digital.

2.13 Arsitektur Chrome Extension (Manifest V3)

Ekstensi Google Chrome adalah perangkat lunak tambahan yang memperluas fungsionalitas peramban dengan beroperasi pada sisi klien (*client-side*). Berdasarkan standar terbaru yakni Manifest V3, ekstensi memiliki kemampuan untuk membaca dan memodifikasi *Document Object Model* (DOM) pada halaman web secara dinamis melalui *Content Scripts* (Ramadhan & Fauzan, 2023). Penggunaan ekstensi pada peramban Chrome sangat efektif untuk melakukan pembatasan atau penyaringan konten secara langsung di sisi pengguna dengan cara memanipulasi elemen halaman web yang sedang dibuka.

Selain untuk fungsi penapisan, arsitektur ekstensi juga mendukung proses pengolahan data yang efisien. Fleksibilitas ini memungkinkan pengembang untuk mengintegrasikan model cerdas ke dalam peramban guna mengeksekusi instruksi tertentu, seperti menyembunyikan elemen komentar yang terdeteksi sebagai konten terlarang secara *real-time*.

Dengan demikian, arsitektur *Chrome Extension* berbasis Manifest V3 merupakan platform yang tepat untuk mengimplementasikan sistem filtrasi cerdas. Kemampuannya dalam berinteraksi langsung dengan DOM dan menjalankan skrip secara *asynchronous* menjadikannya solusi yang efisien untuk melindungi pengguna dari paparan konten spam tanpa membebani performa perangkat secara berlebihan.

2.14 REST API dan Lingkungan FastAPI (Python)

Integrasi arsitektur perangkat lunak antara ekstensi peramban di sisi klien (*client-side*) dengan mesin *Machine Learning* di sisi peladen (*server-side*) memerlukan jalur komunikasi data yang stabil. Konsep *Representational State*

Transfer Application Programming Interface (REST API) digunakan sebagai jembatan pertukaran data ringan berbasis format JSON melalui protokol HTTP. Agar layanan REST API ini dapat melayani permintaan deteksi teks secara instan, diperlukan kerangka kerja (*framework*) yang memiliki kecepatan eksekusi tinggi.

FastAPI merupakan kerangka kerja modern berbasis bahasa pemrograman Python yang dirancang khusus untuk membangun layanan *web service* dengan tingkat performa tinggi (Azhari, 2022). Berbeda dengan Flask yang bersifat *general purpose*, FastAPI dirancang secara eksklusif sebagai *web service* dan mengadopsi mekanisme pemrograman *asynchronous* sehingga mampu memproses permintaan tanpa harus menunggu setiap proses selesai secara berurutan. FastAPI juga memiliki sistem validasi tipe data bawaan yang ketat melalui pustaka Pydantic, yang memastikan setiap data masukan telah tervalidasi sebelum diproses lebih lanjut.

2.15 Unified Modeling Language (UML)

Dalam merancang perangkat lunak yang kompleks, dibutuhkan sebuah pemodelan visual untuk memastikan seluruh alur sistem terekam secara komprehensif sebelum tahapan pengkodean (*coding*) dilakukan. UML (*Unified Modeling Language*) adalah standar bahasa pemodelan berbasis grafik yang digunakan untuk mengidentifikasi kebutuhan sistem (*requirement*), merancang arsitektur analitis, serta mendokumentasikan spesifikasi perangkat lunak yang berorientasi objek (Rosa & Shalahuddin, 2019). Dalam penelitian ini, tiga jenis diagram UML digunakan untuk memodelkan sistem secara komprehensif, yaitu *use case diagram*, *activity diagram*, dan *sequence diagram*.

2.15.1 Use Case Diagram

Use case diagram merupakan bagian dari UML yang merepresentasikan fungsionalitas sistem dari perspektif pengguna (*actor*). Diagram ini tidak menjelaskan secara teknis bagaimana sistem bekerja di balik layar, melainkan menjabarkan batasan sistem (*system boundary*) serta mendeskripsikan operasi dan skenario layanan yang dapat dilakukan aktor terhadap sistem. Komponen utama *use case diagram* terdiri dari *actor* yang digambarkan sebagai ikon manusia, *use case* yang digambarkan sebagai elips berisi nama fungsionalitas, *system boundary* berupa kotak persegi yang membatasi cakupan sistem, serta relasi *association*, *include*, dan *extend* yang menghubungkan komponen-komponen tersebut.

Simbol	Nama	Keterangan
	Actor	Menggambarkan orang, sistem, atau entitas luar yang berinteraksi dengan sistem yang sedang dikembangkan. Aktor tidak terbatas pada manusia saja, melainkan dapat berupa sistem lain atau perangkat yang berkomunikasi dengan sistem.
	Use Case	Menggambarkan fungsionalitas atau layanan yang disediakan oleh sistem kepada aktor. Setiap use case mewakili satu unit fungsional yang dapat digunakan oleh aktor untuk mencapai tujuan tertentu.
	Association	Menggambarkan hubungan atau interaksi antara aktor dengan use case. Garis asosiasi menunjukkan bahwa aktor terlibat dalam pelaksanaan use case tersebut.
	Include	Menunjukkan bahwa sebuah use case selalu menyertakan dan memanggil use case lainnya sebagai bagian dari prosesnya. Relasi ini digunakan untuk menghindari pengulangan fungsi yang sama pada beberapa use case.
	Extend	Menunjukkan bahwa sebuah use case dapat memperluas perilaku use case lainnya secara kondisional. Berbeda dengan include, relasi extend hanya dijalankan jika kondisi tertentu terpenuhi.
	System Boundary	Menggambarkan batas ruang lingkup sistem yang sedang dikembangkan. Semua use case yang berada di dalam kotak ini merupakan bagian dari sistem, sedangkan aktor berada di luar batas sistem.

Gambar 2.4 Simbol-Simbol Use Case Diagram

2.15.2 Activity Diagram

Activity diagram adalah diagram pemodelan dinamis yang menggambarkan alur kerja prosedural (*workflow*) dari setiap proses bisnis di dalam sistem. Diagram ini

secara kronologis memetakan transisi eksekusi kontrol sistem, mulai dari inisialisasi awal, pengambilan keputusan di setiap percabangan kondisi, hingga alur penyelesaian akhir. Komponen utama *activity diagram* meliputi *initial node* berupa lingkaran hitam penuh sebagai titik awal, *action node* berupa persegi panjang berisi nama aksi, *decision node* berupa belah ketupat untuk percabangan kondisi, *fork/join node* berupa garis horizontal untuk alur paralel, serta *activity final node* berupa lingkaran hitam bertepi sebagai titik akhir.

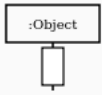
Simbol	Nama	Keterangan
	Initial Node	Menggambarkan titik awal dari sebuah alur aktivitas. Setiap activity diagram hanya memiliki satu initial node yang menandai dimulainya proses atau alur kerja sistem.
	Action Node	Menggambarkan sebuah aksi atau aktivitas yang dilakukan dalam sistem. Setiap action node merepresentasikan satu langkah proses yang tidak dapat dibagi lebih lanjut dalam konteks diagram tersebut.
	Decision Node	Menggambarkan titik percabangan kondisional dalam alur aktivitas. Dari satu decision node dapat keluar dua atau lebih alur yang berbeda berdasarkan kondisi atau syarat tertentu yang dievaluasi.
	Fork / Join Node	Menggambarkan pemisahan alur menjadi beberapa alur paralel (fork) atau penggabungan beberapa alur paralel menjadi satu alur (join). Keduanya digambarkan dengan garis tebal horizontal atau vertikal.
	Control Flow	Menggambarkan arah alur atau transisi antar aktivitas. Anak panah menunjukkan urutan pelaksanaan aktivitas dari satu action node ke action node berikutnya.
	Activity Final Node	Menggambarkan titik akhir dari seluruh alur aktivitas dalam diagram. Ketika alur mencapai activity final node, seluruh proses dalam diagram dinyatakan selesai.
	Swimlane	Mengelompokkan aktivitas berdasarkan aktor atau komponen yang bertanggung jawab atas pelaksanaannya. Swimlane memudahkan pembaca untuk memahami siapa yang melakukan setiap aktivitas dalam diagram.

Gambar 2.5 Simbol-Simbol Activity Diagram

2.15.3 Sequence Diagram

Sequence diagram adalah diagram UML yang menggambarkan interaksi antar objek atau komponen sistem berdasarkan urutan waktu (*time-ordered*). Diagram ini sangat efektif untuk memodelkan komunikasi antar lapisan sistem (*layer*) yang berbeda, seperti interaksi antara ekstensi peramban, server API, dan model

klasifikasi. Komponen utama *sequence diagram* meliputi *lifeline* berupa garis putus-putus vertikal yang merepresentasikan objek atau komponen, *activation bar* berupa persegi panjang tipis di atas *lifeline* yang menunjukkan periode aktif suatu objek, serta *message* berupa anak panah horizontal yang merepresentasikan komunikasi antar *lifeline*, baik berupa *synchronous message* (panah penuh), *return message* (panah putus-putus), maupun *asynchronous message* (panah terbuka).

Simbol	Nama	Keterangan
	Lifeline	Menggambarkan sebuah objek atau komponen sistem yang berpartisipasi dalam interaksi. Terdiri dari kotak di bagian atas yang berisi nama objek dan garis putus-putus vertikal di bawahnya yang merepresentasikan keberadaan objek sepanjang waktu.
	Activation Bar	Menggambarkan periode waktu saat sebuah objek sedang aktif menjalankan proses atau operasi. Digambarkan sebagai persegi panjang tipis yang berada di atas lifeline, dengan panjang yang sebanding dengan durasi aktivitas objek tersebut.
	Synchronous Message	Menggambarkan pesan yang dikirimkan dari satu objek ke objek lain di mana pengirim harus menunggu hingga penerima selesai memproses sebelum melanjutkan eksekusi. Digambarkan dengan anak panah padat.
	Return Message	Menggambarkan pesan balasan yang dikirimkan kembali dari objek penerima kepada objek pengirim setelah sebuah operasi selesai diproses. Digambarkan dengan anak panah putus-putus yang mengarah kembali ke pemanggil.
	Asynchronous Message	Menggambarkan pesan yang dikirimkan dari satu objek ke objek lain di mana pengirim tidak perlu menunggu respons dan dapat langsung melanjutkan eksekusi. Digunakan untuk komunikasi yang berjalan secara paralel atau non-blocking.
	Combined Fragment	Menggambarkan blok interaksi bersyarat atau berulang dalam sequence diagram. Jenis yang umum digunakan antara lain loop (pengulangan), alt (kondisi alternatif/if-else), dan opt (kondisi opsional).

Gambar 2.6 Simbol-Simbol Sequence Diagram

2.16 Metode Analisis Sistem (PIECES)

Untuk mengukur tingkat kelayakan sistem yang dikembangkan, digunakan kerangka kerja analisis PIECES. Menurut Kinanti & Indriyanti (2021), analisis PIECES adalah metode evaluasi sistem yang meninjau permasalahan dan peluang perbaikan dari enam aspek utama, yaitu *Performance* (Kinerja), *Information*

(Informasi), *Economics* (Ekonomi), *Control* (Pengendalian), *Efficiency* (Efisiensi), dan *Service* (Pelayanan). Keenam aspek tersebut dijabarkan pada tabel berikut.

Tabel 2.1 Aspek-Aspek Analisis PIECES

Aspek	Penjelasan
<i>Performance</i> (Kinerja)	Mengukur kemampuan sistem dalam memproses sejumlah pekerjaan dalam satuan waktu tertentu. Kinerja diukur melalui dua parameter utama: <i>throughput</i> , yaitu jumlah pekerjaan yang berhasil diselesaikan sistem dalam periode tertentu, dan <i>response time</i> , yaitu rata-rata waktu yang dibutuhkan sistem untuk merespons sebuah permintaan.
<i>Information</i> (Informasi)	Mengevaluasi kualitas informasi yang dihasilkan sistem dari tiga dimensi utama: akurasi (<i>accurate</i>), yang berarti informasi bebas dari kesalahan; ketepatan waktu (<i>timely</i>), yang berarti informasi tersedia saat dibutuhkan; dan relevansi (<i>relevant</i>), yang berarti informasi sesuai dengan kebutuhan pengguna.
<i>Economics</i> (Ekonomi)	Menilai efisiensi biaya operasional sistem, mencakup analisis pengurangan pengeluaran yang tidak perlu dan peningkatan manfaat yang diperoleh dari sistem yang dikembangkan dibandingkan dengan kondisi sebelumnya.

Control (Pengendalian)	Menilai kemampuan sistem dalam mencegah atau mendeteksi kesalahan operasional, menjaga keamanan data dari akses yang tidak sah, serta memastikan keandalan sistem dalam menghadapi potensi gangguan atau kerusakan.
<i>Efficiency</i> (Efisiensi)	Mengukur sejauh mana sistem dapat memaksimalkan keluaran (<i>output</i>) dengan menggunakan sumber daya masukan (<i>input</i>) seminimal mungkin, sehingga tidak terjadi pemborosan waktu, tenaga, maupun biaya operasional.
<i>Service</i> (Pelayanan)	Menilai kemudahan dan kenyamanan yang diberikan sistem kepada pengguna dalam menyelesaikan tugasnya, termasuk keramahan antarmuka, keandalan layanan, serta kemampuan sistem dalam memenuhi kebutuhan pengguna secara konsisten.

2.17 Studi Penelitian Terdahulu

Kajian terhadap studi penelitian terdahulu dilakukan untuk mengetahui perkembangan penelitian yang telah ada, mengidentifikasi celah penelitian (*research gap*), serta menjadi landasan bagi pengembangan sistem yang diusulkan. Penelitian ini merujuk pada dua studi penelitian terdahulu yang memiliki tingkat relevansi paling tinggi terhadap deteksi komentar spam promosi judi *online* di

platform YouTube menggunakan *Machine Learning*. Berikut adalah uraian dari kedua penelitian tersebut:

1. **Angelo, Robet & Hendrik (2025)**, *Comparative Performance of Machine Learning Algorithms for Detecting Online Gambling Promotional Comments on YouTube*.

- a. Input: Dataset komentar YouTube yang mengandung promosi judi *online* berbahasa Indonesia, termasuk komentar yang menggunakan teknik *obfuscation* seperti karakter Unicode, emoji, spasi tidak beraturan, dan simbol untuk menghindari moderasi otomatis. Dataset diperluas menggunakan teknik *semi-supervised pseudo-labelling* dari 1.648 menjadi 9.111 komentar.
- b. Proses: Membandingkan performa empat algoritma *Machine Learning* Multinomial Naive Bayes, Logistic Regression, Random Forest, dan SVM menggunakan pipeline TF-IDF dengan normalisasi karakter khusus, *stopword removal*, dan stemming Nazief–Adriani. Optimasi hiperparameter dan penyeimbangan kelas dilakukan melalui SMOTE.
- c. Output: Seluruh model mencapai akurasi di atas 98% pada set pengujian. SVM memberikan performa paling seimbang dengan *F1-score* tertinggi sebesar 0,9908 untuk kelas promosi, sehingga direkomendasikan untuk deployment operasional dalam moderasi otomatis komentar judi *online*.

2. **Airlangga (2024a)**, *Spam Detection on YouTube Comments Using Advanced Machine Learning Models: A Comparative Study*

- a. Input: Himpunan data (*dataset*) komentar YouTube yang telah dikategorikan secara manual ke dalam dua kelas utama, yaitu spam dan *ham* (normal).
- b. Proses: Menjalankan studi perbandingan performa klasifikasi menggunakan model *Machine Learning* tingkat lanjut, yang meliputi algoritma Linear SVM, Random Forest, dan XGBoost.
- c. Output: Hasil evaluasi membuktikan bahwa algoritma Linear SVM memberikan performa yang paling stabil dan unggul dengan tingkat akurasi klasifikasi mencapai 95%.

Berdasarkan kedua penelitian terdahulu di atas, dapat disimpulkan bahwa algoritma SVM secara konsisten menunjukkan performa terbaik dalam mendeteksi komentar spam dan promosi judi *online* di YouTube. Namun demikian, kedua penelitian tersebut menghasilkan model yang beroperasi secara *offline* model hanya dapat digunakan melalui skrip atau antarmuka terpisah, bukan langsung terintegrasi ke dalam pengalaman berselancar pengguna. Oleh karena itu, terdapat celah penelitian berupa kebutuhan akan sistem yang mampu menjalankan deteksi spam secara *real-time* langsung di peramban pengguna, tanpa memerlukan intervensi manual, melalui implementasi model SVM pada arsitektur *Chrome Extension*.

2.18 Persyaratan Sistem Konseptual

Berdasarkan hasil analisis terhadap studi penelitian terdahulu, sistem yang dirancang dalam penelitian ini harus memenuhi persyaratan konseptual tertentu guna menutupi celah kekurangan pada penelitian sebelumnya. Berikut adalah rincian persyaratan sistem konseptual tersebut:

1. Modul Prapemrosesan Teks dengan Teknik *String Normalization*

- a. Input: Data teks komentar dari platform YouTube yang mengandung berbagai variasi penyamaran karakter (*obfuscation*), seperti penggunaan *homoglyphs*, simbol Unicode khusus, dan karakter non-standar yang digunakan untuk menghindari moderasi otomatis.
- b. Proses: Mendeteksi karakter non-ASCII dan melakukan konversi teks menggunakan teknik *string normalization* untuk mengembalikan karakter manipulatif ke bentuk teks standar sebelum diteruskan ke tahap ekstraksi fitur.
- c. Output: Teks komentar yang bersih dan ternormalisasi, siap diproses oleh tahap ekstraksi fitur TF-IDF dan klasifikasi SVM.
- d. Pengembangan (*Improvement*): Angelo dkk. (2025) menunjukkan bahwa komentar promosi judi *online* di YouTube secara aktif menggunakan teknik *obfuscation* berbasis Unicode untuk menghindari deteksi. Penelitian ini mengadopsi pendekatan normalisasi karakter serupa dan mengintegrasikannya ke dalam pipeline prapemrosesan yang berjalan secara otomatis di sisi klien.

2. Modul Deteksi Real-Time Berbasis Chrome Extension

- a. Input: Elemen *Document Object Model* (DOM) pada halaman YouTube yang memuat teks komentar pengguna secara dinamis.
- b. Proses: Menggunakan *content script* untuk membaca teks komentar secara *asynchronous*, mengirimkannya ke model SVM melalui REST API FastAPI, dan menerima hasil klasifikasi untuk diterapkan pada tampilan halaman.

- c. Output: Perubahan status tampilan pada elemen komentar yang teridentifikasi sebagai spam judi *online* melalui manipulasi CSS (`display: none`), sehingga komentar tersebut tersembunyi secara visual tanpa mengganggu konten lainnya.
- d. Pengembangan (*Improvement*): Airlangga (2024a) membuktikan akurasi Linear SVM mencapai 95% dalam klasifikasi komentar spam YouTube, namun implementasinya terbatas pada evaluasi data secara *offline*. Penelitian ini mengembangkan sistem tersebut dengan mengintegrasikan model SVM ke dalam arsitektur *Chrome Extension* sehingga perlindungan dapat dirasakan langsung oleh pengguna secara *real-time* tanpa intervensi manual.

3. Keandalan Klasifikasi dengan Algoritma SVM

- a. Input: Fitur numerik hasil ekstraksi *Term Frequency-Inverse Document Frequency* (TF-IDF) dari teks komentar yang telah melalui tahap normalisasi.
- b. Proses: Melakukan pemisahan data menggunakan *hyperplane* optimal yang dibentuk oleh algoritma Linear SVM untuk menentukan label akhir setiap komentar.
- c. Output: Keputusan klasifikasi biner berupa label "Spam" atau "Ham" (normal) dengan tingkat presisi dan *recall* yang tinggi.
- d. Pengembangan (*Improvement*): Sistem yang dibangun secara khusus menargetkan komentar promosi judi *online* berbahasa Indonesia sebuah domain yang belum menjadi fokus utama penelitian Airlangga (2024a) maupun penelitian sebelumnya sehingga model dilatih menggunakan

dataset yang dikumpulkan dan dilabeli secara manual dari platform YouTube untuk memastikan relevansi terhadap konteks bahasa dan konten yang spesifik.

BAB III

OBJEK PENELITIAN

3.1 Tinjauan Umum Objek Penelitian

Objek penelitian pada skripsi ini difokuskan pada pengembangan perangkat lunak berbasis klien (*client-side*), yaitu berupa ekstensi peramban web (*browser extension*) Google Chrome. Perangkat lunak ini dirancang secara khusus untuk melakukan deteksi dan penyembunyian secara instan terhadap komentar yang bermuatan promosi judi *online* pada antarmuka platform YouTube. Pendekatan ini dipilih untuk memberikan lapisan keamanan tambahan yang independen bagi pengguna, tanpa perlu menunggu intervensi dari administrator platform utama.

3.1.1 Behaviour (Tingkah Laku) Objek Penelitian

Tingkah laku (*behaviour*) dari objek penelitian ini bertumpu pada interaksi langsung dan dinamis antara ekstensi peramban dengan *Document Object Model* (DOM) pada halaman web YouTube yang sedang dimuat oleh pengguna. Perangkat lunak ini beroperasi secara *asynchronous* di latar belakang untuk meminimalisasi beban komputasi peramban. Secara teknis, alur tingkah laku sistem dapat diuraikan melalui tahapan operasional berikut:

1. Inisiasi dan Pemantauan Elemen (DOM Observation)

Ekstensi memanfaatkan antarmuka *MutationObserver* bawaan peramban untuk memantau setiap perubahan pada struktur DOM. Saat pengguna menggulir halaman dan komentar baru dimuat secara dinamis oleh YouTube (melalui *AJAX/Fetch*), ekstensi secara otomatis mendeteksi *node* teks baru tersebut tanpa perlu memuat ulang halaman.

2. Ekstraksi dan Prapemrosesan Klien (*String Normalization*)

Teks komentar yang berhasil ditangkap kemudian diekstrak dan langsung diproses secara lokal. Modul ini menjalankan algoritma *String Normalization* menggunakan reguler ekspresi (*RegEx*) untuk membersihkan karakter pengganggu, serta memetakan dan mengonversi variasi karakter Unicode khusus (seperti huruf Cyrillic atau simbol yang menyerupai alfabet Latin) kembali menjadi format teks ASCII standar.

3. Transmisi Data Asynchronous

Teks yang telah dinormalisasi dikemas dalam format *JavaScript Object Notation* (JSON) dan dikirimkan secara *real-time* menuju peladen API (*backend server*) menggunakan metode *HTTP POST request* tanpa mengganggu pengalaman menonton pengguna.

4. Klasifikasi pada Peladen (*Model Evaluation*)

Peladen berbasis Python menggunakan kerangka kerja FastAPI menerima data teks dan mengeksekusi ekstraksi fitur lanjutan menggunakan TF-IDF. Representasi numerik tersebut kemudian dimasukkan ke dalam model *Support Vector Machine* (SVM) yang telah dilatih sebelumnya, guna menentukan garis pemisah (*hyperplane*) dan menghasilkan label klasifikasi (Spam atau Normal).

5. Manipulasi Antarmuka Pengguna (*DOM Manipulation*)

Peladen mengembalikan hasil prediksi ke ekstensi Chrome. Apabila label yang dikembalikan adalah "Spam", Content Script dari ekstensi akan mencari ID elemen komentar yang bersangkutan dan menerapkan injeksi kode properti *Cascading Style Sheets* (CSS) dengan parameter *display: none* atau *visibility:*

hidden. Hal ini secara instan menghilangkan komentar tersebut dari pandangan pengguna dalam peramban web yang digunakan.

Tingkah laku dari objek penelitian ini memiliki keunggulan yang signifikan, yaitu kemampuannya memberikan perlindungan proaktif yang disesuaikan secara lokal di komputer pengguna, sehingga tidak memerlukan modifikasi pada arsitektur basis data (*database*) milik YouTube. Namun, batasan dari tingkah laku ini mencakup tingkat ketergantungan yang tinggi terhadap ketersediaan koneksi internet yang stabil untuk mengakses peladen API, serta kerentanan jika pihak YouTube melakukan perombakan besar-besaran pada struktur *tag* HTML dan penamaan *class* DOM mereka.

3.1.2 Struktur/Kategori Objek Penelitian

Objek penelitian ini dikategorikan sebagai perangkat lunak filtrasi konten berbasis peramban dengan standar arsitektur pengembangan Manifest V3. Guna mendukung proses deteksi berbasis Machine Learning yang berat, struktur sistem ini mengusung pola arsitektur terdistribusi yang memisahkan beban kerja antara sisi klien dan sisi peladen. Struktur objek penelitian ini terdiri dari:

1. Sisi Klien (Ekstensi Chrome)

Bertindak sebagai agen interaktif di sisi pengguna. Terdiri dari Manifest File (*manifest.json*) yang mengatur hak akses permission peramban, *Content Scripts* yang diinjeksi langsung ke halaman YouTube untuk membaca teks dan mengeksekusi manipulasi DOM, serta *Service Workers* (*background.js*) yang mengelola siklus hidup ekstensi dan menangani rute komunikasi ke peladen luar secara efisien.

2. Sisi Peladen (Backend API)

Bertindak sebagai pusat pemrosesan cerdas. Lingkungan peladen dibangun menggunakan bahasa pemrograman Python dengan kerangka kerja FastAPI, yang mengadopsi mekanisme pemrograman *asynchronous* sehingga sangat optimal untuk menangani permintaan teks secara bersamaan dari ekstensi. Peladen ini memuat *pipeline* prapemrosesan teks, ekstraksi fitur TF-IDF, dan model *Support Vector Machine* (SVM) yang telah dilatih, serta menyediakan dua endpoint REST API yaitu */predict* untuk klasifikasi tunggal dan */predict/batch* untuk klasifikasi hingga 50 komentar sekaligus.

3.2 Sistem Berjalan

3.2.1 Tinjauan Umum

Sistem penyaringan komentar yang berjalan saat ini pada platform YouTube pada dasarnya beroperasi melalui mekanisme hibrida, yang menggabungkan filtrasi otomatis berbasis kamus kata kunci (*keyword-based filtering / regular expressions blacklist*) dan sistem moderasi manual berbasis pelaporan dari komunitas (*flagging*). YouTube secara terpusat menyaring atau menahan komentar yang secara eksplisit memuat kata-kata kasar, tautan URL ke situs berbahaya, atau kosakata spesifik yang telah dimasukkan ke dalam konfigurasi daftar hitam oleh kreator konten pada panel YouTube Studio.

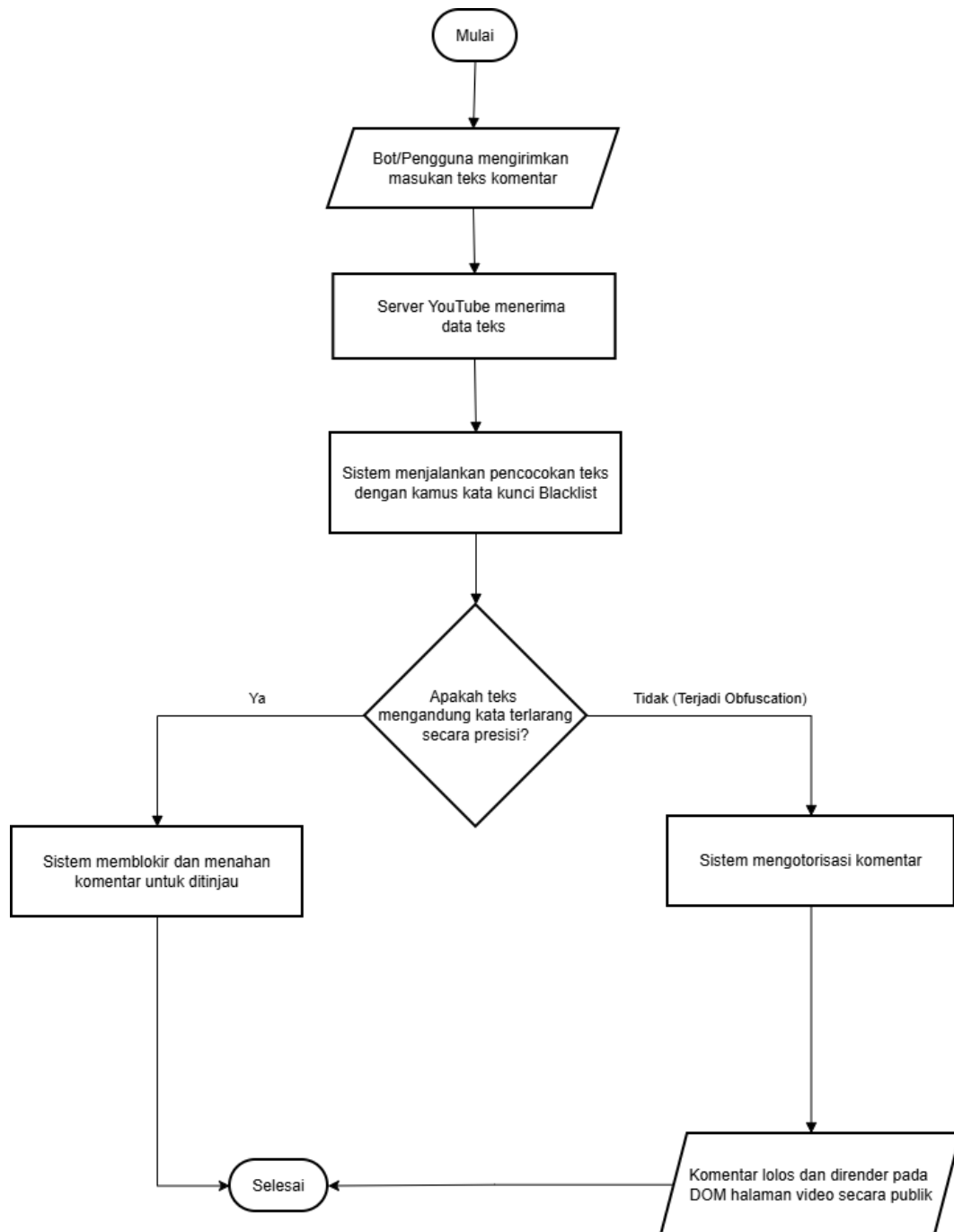
3.2.2 Analisis Proses Sistem Berjalan

Walaupun sistem filter bawaan platform ini mampu beroperasi pada skala besar (jutaan komentar per detik), proses identifikasi spam konvensional ini memiliki celah struktural dalam menangani analisis semantik tingkat lanjut, khususnya

terhadap teknik penyamaran teks manipulatif (*obfuscation*). Analisis terhadap alur kelemahan sistem berjalan dapat diuraikan melalui tahapan berikut:

1. Pelaku serangan (*spammer*) menggunakan skrip otomatis untuk menghasilkan teks promosi judi *online* yang telah dimanipulasi secara visual. Manipulasi ini memanfaatkan teknik substitusi karakter (*homoglyphs*), contohnya mengganti alfabet Latin "O" dengan angka nol "0" atau huruf Yunani "Omicron", serta menyisipkan tanda baca acak di antara susunan kata (contoh: "S.L.O.T", "J-u-d-i").
2. Komentar manipulatif tersebut dikirimkan ke pangkalan data server YouTube melalui protokol standar aplikasi.
3. Mesin filtrasi bawaan platform menjalankan pencocokan *string* linier dengan kamus kata kunci terlarang (misalnya: mencari kata "slot").
4. Karena susunan *byte* dari karakter-karakter manipulatif tersebut (secara Unicode) berbeda dengan kamus dasar, algoritma platform gagal mengidentifikasinya sebagai leksikon yang melanggar kebijakan. Platform menganggap susunan tersebut sebagai teks acak yang tidak bermakna atau tidak membahayakan.
5. Komentar dinyatakan berstatus valid dan diotorisasi untuk ditampilkan pada antarmuka publik. Akibatnya, promosi ilegal tersebut berhasil menembus lapisan keamanan dan terpapar kepada jutaan pengguna.

3.2.3 Flowchart Sistem Berjalan



Gambar 3.1 Flowchart Sistem Berjalan

Berikut ini adalah penjelasan dari diagram flowchart sistem berjalan:

1. Mulai (*Start*)
2. Entitas pengguna atau bot mengirimkan masukan berupa teks komentar ke server YouTube.
3. Sistem YouTube menerima permintaan dan menjalankan proses pencocokan dengan pangkalan data kata kunci (*Keyword Blacklist Checking*).
4. Titik Keputusan (*Decision*): Apakah teks komentar mengandung entitas kata terlarang secara presisi?
5. Apabila kondisi bernilai "Ya", sistem menjalankan proses pemblokiran dan menyembunyikan komentar ke peninjauan (*Hold for Review*).
6. Apabila kondisi bernilai "Tidak" (kata disamarkan dengan *obfuscation* yang tidak terdaftar), sistem menjalankan proses otorisasi.
7. Komentar lolos dan dirender pada DOM halaman video secara publik.
8. Selesai (*End*).

3.3 Permasalahan Sistem (PIECES)

Untuk mengukur keandalan dan mendiagnosis secara presisi kelemahan yang ada pada sistem berjalan, penelitian ini menggunakan metode evaluasi PIECES (*Performance, Information, Economics, Control, Efficiency, dan Service*). Berikut adalah penjabaran analisis permasalahan dari sistem pemfilteran YouTube saat ini:

1. Analisis Kinerja (*Performance*)

Kinerja infrastruktur YouTube sangat mumpuni dalam memproses dan menyaring jutaan komentar teks baku per detik. Namun, *throughput* (kemampuan menyelesaikan pekerjaan) sistem ini menurun drastis ketika berhadapan dengan teks yang dimanipulasi (*obfuscation*). Kegagalan sistem

dalam membedah karakter manipulatif secara instan mengakibatkan latensi tinggi dalam penanganan spam, karena pada akhirnya platform harus bergantung pada proses moderasi (*flagging*) manual oleh pengguna yang membutuhkan waktu respon (*response time*) sangat lambat.

2. Analisis Informasi (*Information*)

Sistem berjalan gagal mempertahankan kualitas informasi yang disajikan di ruang publik. Lemahnya pemfilteran menyebabkan kolom komentar dibanjiri oleh teks dan tautan yang sama sekali tidak relevan dengan konteks video. Informasi promosi judi *online* yang lolos dari sistem ini sangat menyesatkan, mereduksi nilai edukasi atau hiburan dari konten utama, dan mendegradasi kualitas informasi platform secara keseluruhan.

3. Analisis Ekonomi (*Economics*)

Dari sudut pandang makro, lolosnya promosi perjudian ilegal ini menimbulkan ancaman kerugian finansial yang masif bagi masyarakat yang terjerat. Sementara dari sisi operasional platform, mempekerjakan ribuan moderator manusia untuk menyaring sisa komentar spam yang gagal ditangkap oleh mesin otomatis merupakan pemborosan anggaran (inefisiensi biaya) yang seharusnya dapat ditekan dengan sistem cerdas di sisi klien.

4. Analisis Kendali (*Control*)

Tingkat pengendalian (*control*) sistem berjalan terbukti sangat lemah terhadap serangan manipulasi *String*. Ketiadaan mekanisme *String Normalization* yang agresif membuat sistem kehilangan kendali saat berhadapan dengan *homoglyphs*, penyisipan tanda baca, atau penggunaan variasi fon Unicode khusus. Celah keamanan ini mempermudah pihak tak bertanggung jawab

menyisipkan *Uniform Resource Locator* (URL) berbahaya yang mengancam privasi dan keamanan data pengguna.

5. Analisis Efisiensi (*Efficiency*)

Mekanisme pemfilteran bawaan yang mengandalkan pembaruan daftar hitam (*blacklist*) kata kunci secara manual sangat tidak efisien. *Spammer* hanya perlu mengubah satu struktur karakter (misalnya "SLOT" menjadi "5L0T") untuk menembus filter. Pola ini memaksa administrator untuk terus memperbarui kamus data secara konstan, sebuah proses yang menghabiskan waktu dan tenaga, serta selalu tertinggal selangkah dari laju otomatisasi bot penyerang.

6. Analisis Pelayanan (*Service*)

Dampak akumulatif dari kelemahan sistem berjalan bermuara pada buruknya tingkat pelayanan dan pengalaman pengguna (*user experience*). Kegagalan platform dalam menciptakan lingkungan digital yang aman dari paparan promosi ilegal memicu ketidaknyamanan berinteraksi, mematikan diskusi yang konstruktif di kolom komentar, dan menurunkan tingkat kepercayaan (*trust*) pengguna terhadap kredibilitas layanan platform tersebut.

Tabel 3.1 Ringkasan Analisis PIECES Sistem Berjalan

Aspek PIECES	Identifikasi Masalah	Solusi yang Diusulkan
Analisis Kinerja (<i>Performance</i>)	<i>Throughput</i> sistem menurun drastis dan latensi memanjang jika berhadapan dengan komentar yang menggunakan teknik	Penerapan normalisasi teks otomatis di sisi klien sebelum proses klasifikasi dimulai.

	penyamaran teks (<i>obfuscation</i>).	
Analisis Informasi (<i>Information</i>)	Informasi promosi judi <i>online</i> yang lolos mereduksi kualitas konten, menyesatkan publik, dan mendegradasi nilai edukasi/hiburan platform.	Implementasi model klasifikasi berbasis <i>machine learning</i> untuk mengidentifikasi dan menyembunyikan konten menyesatkan secara otomatis.
Analisis Ekonomi (<i>Economics</i>)	Lolosnya spam memicu risiko kerugian finansial di masyarakat, serta inefisiensi biaya operasional karena masih bergantung pada moderator manusia.	Pengembangan sistem deteksi otomatis di sisi klien yang dapat diakses secara gratis tanpa biaya berlangganan layanan moderasi berbayar.
Analisis Kendali (<i>Control</i>)	Lemahnya kendali sistem (<i>String Normalization</i> yang tidak agresif) menyebabkan platform gagal memetakan karakter Unicode khusus dan URL berbahaya.	Penerapan tujuh tahap normalisasi teks termasuk konversi NFKC Unicode, penanganan <i>homoglyph</i> lintas skrip, dan kanonisasi nama <i>brand</i> judi <i>online</i> memperkuat lapisan kendali deteksi.
Analisis Efisiensi (<i>Efficiency</i>)	Pembaruan kamus <i>blacklist</i> secara manual sangat tidak efisien dan selalu tertinggal dari laju pembaruan otomatisasi skrip <i>bot spammer</i> .	Penerapan lapisan normalisasi karakter yang agresif untuk memperkuat kendali deteksi terhadap berbagai teknik penyamaran teks.

Analisis Pelayanan (<i>Service</i>)	Paparan promosi ilegal menurunkan kenyamanan (<i>user experience</i>), merusak diskusi konstruktif, dan mendegradasi kredibilitas layanan platform.	Penyembunyian komentar spam secara otomatis di peramban pengguna untuk menciptakan pengalaman berselancar yang lebih bersih dan aman.
--	---	---

Berdasarkan hasil analisis menggunakan metode PIECES terhadap sistem yang berjalan, dapat disimpulkan bahwa kelemahan paling fundamental terletak pada aspek Kendali (*Control*) dan Efisiensi (*Efficiency*). Ketidakmampuan sistem *native* platform dalam mengurai teks yang dimanipulasi (*obfuscation*) menyebabkan efisiensi pemfilteran menurun dan hilangnya kendali terhadap penyebaran promosi perjudian. Kegagalan pada dua aspek inti ini secara berantai berdampak buruk pada Kinerja (*Performance*) moderasi, merusak validitas Informasi (*Information*), memicu kerugian Ekonomi (*Economics*), dan pada akhirnya mendegradasi kualitas Pelayanan (*Service*) kepada pengguna.

Oleh karena itu, sebagai solusi ilmiah dan teknis atas permasalahan tersebut, sangat mendesak untuk dirancang sebuah sistem usulan berupa ekstensi peramban Google Chrome. Sistem usulan ini diproyeksikan mampu menutupi celah kendali platform dengan mengimplementasikan teknik *String Normalization* langsung di sisi klien (*client-side*), serta meningkatkan efisiensi deteksi secara signifikan melalui integrasi model kecerdasan buatan berbasis *Support Vector Machine* (SVM). Kombinasi teknologi ini akan menciptakan benteng pertahanan yang proaktif, adaptif, dan mampu menyembunyikan ancaman spam secara *real-time* sebelum terpapar kepada pengguna.

BAB IV

PERANCANGAN SISTEM

4.1 Analisis Kebutuhan Sistem

Analisis kebutuhan sistem merupakan tahap awal dalam perancangan yang bertujuan mengidentifikasi secara menyeluruh apa yang harus mampu dilakukan oleh sistem yang diusulkan. Pada penelitian ini, sistem yang dirancang adalah ekstensi peramban Google Chrome yang bekerja secara *real-time* untuk mendeteksi dan menyembunyikan komentar promosi judi *online* pada platform YouTube menggunakan model klasifikasi *Support Vector Machine* (SVM).

4.1.1 Kebutuhan Fungsional

Kebutuhan fungsional mendeskripsikan fungsi-fungsi utama yang harus dapat dijalankan oleh sistem. Kebutuhan fungsional sistem ini adalah sebagai berikut:

1. Sistem harus mampu mendeteksi komentar baru yang dimuat secara dinamis pada halaman YouTube tanpa memerlukan pemuatan ulang halaman.
2. Sistem harus mampu mengirimkan teks komentar ke server klasifikasi melalui jaringan internet dan menerima hasilnya secara *real-time*.
3. Sistem harus mampu mengklasifikasikan komentar sebagai spam atau non-spam berdasarkan model SVM yang telah dilatih.
4. Sistem harus mampu menyembunyikan komentar yang terdeteksi sebagai spam secara otomatis dari tampilan halaman YouTube sesuai mode yang dipilih pengguna.
5. Sistem harus menyediakan antarmuka *popup* yang menampilkan status koneksi server dan statistik jumlah komentar yang telah dipindai dan disembunyikan.

6. Sistem harus menyediakan dua mode penyembunyian yang dapat dipilih pengguna: mode Redupkan yang menampilkan komentar spam dengan transparansi 15% disertai *badge* persentase kepercayaan model yang dapat diklik untuk memperlihatkan komentar kembali, dan mode Hilangkan yang menyembunyikan komentar sepenuhnya dari tampilan.
7. Sistem harus menyediakan pengaturan ambang batas kepercayaan (*confidence threshold*) yang dapat disesuaikan oleh pengguna melalui *popup*.
8. Sistem harus mampu menerima koreksi dari pengguna ketika terjadi kesalahan klasifikasi melalui tombol "Bukan spam?" yang tersedia pada mode Redupkan.

4.1.2 Kebutuhan Non-Fungsional

Kebutuhan non-fungsional mendeskripsikan batasan dan standar kualitas yang harus dipenuhi sistem di luar fungsi utamanya.

1. Performa

Sistem harus mampu memproses dan mengklasifikasikan hingga 50 komentar dalam satu permintaan dengan latensi yang tidak mengganggu pengalaman berselancar pengguna.

2. Ketersediaan

Sistem harus dapat berjalan secara otomatis setiap kali pengguna membuka halaman YouTube selama ekstensi aktif dan server dapat diakses melalui jaringan internet

3. Keandalan

Sistem harus tetap berfungsi dengan baik meskipun terjadi kegagalan koneksi ke server, dengan cara menampilkan status *offline* tanpa menyebabkan peramban berhenti bekerja.

4. Keamanan

Server API di-*deploy* pada infrastruktur *Virtual Private Server* (VPS) yang terpisah dari perangkat pengguna, sehingga model dan pipeline klasifikasi tidak dapat diakses atau dimodifikasi secara langsung oleh pengguna akhir.

5. Kemudahan Penggunaan

Sistem beroperasi secara otomatis di latar belakang tanpa memerlukan interaksi manual dari pengguna untuk setiap komentar yang diproses.

4.1.3 Kebutuhan Perangkat Lunak

Kebutuhan perangkat lunak mencakup seluruh perangkat lunak yang diperlukan untuk menjalankan sistem, baik pada sisi pengembangan maupun sisi pengguna akhir.

Tabel 4.1 Kebutuhan Perangkat Lunak

No.	Perangkat Lunak	Fungsi
1	Google Chrome (versi 88 ke atas)	Peramban yang mendukung Manifest V3 untuk menjalankan ekstensi
2	Python 3.9 ke atas	<i>Runtime server</i> FastAPI dan <i>pipeline machine learning</i>
3	FastAPI	Kerangka kerja REST API untuk server klasifikasi
4	Scikit-learn	Pustaka untuk pipeline TF-IDF dan model SVM
5	Ubuntu Server 24.04 LTS	Sistem operasi VPS untuk menjalankan server

6	Uvicorn	<i>ASGI server</i> untuk menjalankan FastAPI di VPS
7	Node.js	<i>Runtime</i> untuk skrip <i>scraper</i> pengumpulan data
8	Joblib	Pustaka untuk menyimpan dan memuat model terlatih

4.1.4 Kebutuhan Perangkat Keras

Kebutuhan perangkat keras mencakup spesifikasi minimum yang diperlukan untuk menjalankan sistem secara optimal, baik pada sisi pengembangan maupun sisi pengguna.

Tabel 4.2 Kebutuhan Perangkat Keras

No.	Komponen	Spesifikasi
1	Processor (VPS)	2 vCPU
2	RAM (VPS)	2 GB
3	Penyimpanan (VPS)	40 GB SSD
4	Processor (Klien)	Intel Core i3 atau setara, 1.8 Ghz
5	RAM (Klien)	4 GB
6	Koneksi Jaringan	Internet untuk komunikasi ekstensi ke VPS

4.2 Metode Pengembangan Sistem

Penelitian ini menggunakan metode *Waterfall* dalam proses pengembangan sistemnya. Penjelasan konseptual mengenai metode ini telah diuraikan pada Sub-bab 2.9. Penerapan kelima tahapan *Waterfall* dalam konteks penelitian ini adalah sebagai berikut:

1. Analisis Kebutuhan (*Requirements Analysis*)

Tahap ini mencakup identifikasi seluruh kebutuhan fungsional dan non-fungsional sistem yang telah diuraikan pada Sub-bab 4.1. Kebutuhan

fungsi-fungsional meliputi kemampuan sistem dalam mendeteksi komentar spam judi *online* secara *real-time*, melakukan normalisasi karakter Unicode, mengklasifikasikan teks menggunakan model SVM, serta menyembunyikan komentar yang terdeteksi sebagai spam melalui manipulasi CSS. Kebutuhan non-fungsional mencakup *response time* yang rendah agar tidak mengganggu pengalaman pengguna saat menjelajahi YouTube.

2. Perancangan Sistem (*System Design*)

Tahap ini mencakup perancangan arsitektur sistem secara menyeluruh, meliputi desain pipeline prapemrosesan teks, konfigurasi ekstraksi fitur TF-IDF, pemilihan parameter model SVM, perancangan *endpoint* REST API pada FastAPI, serta perancangan alur komunikasi antara *Chrome Extension* dan server. Perancangan sistem divisualisasikan melalui diagram UML yang mencakup *use case diagram*, *activity diagram*, dan *sequence diagram* yang diuraikan lebih lanjut pada sub-bab perancangan sistem di bab ini.

3. Implementasi (*Implementation*)

Pada tahap ini seluruh rancangan diimplementasikan ke dalam kode program. Implementasi dibagi menjadi dua komponen utama: sisi server menggunakan Python dengan pustaka Scikit-learn untuk pipeline TF-IDF dan SVM serta FastAPI sebagai kerangka kerja REST API yang di-*deploy* pada *Virtual Private Server* (VPS), dan sisi klien menggunakan JavaScript dengan standar arsitektur Manifest V3 untuk *Chrome Extension*.

4. Pengujian (*Testing*)

Tahap ini mencakup evaluasi performa model klasifikasi menggunakan metrik akurasi, presisi, *recall*, dan F1-Score terhadap data uji. Selain itu

dilakukan pengujian komparasi antara tiga algoritma — Linear SVM, *Multinomial Naive Bayes*, dan *Logistic Regression* — untuk membuktikan secara empiris keunggulan SVM sebagai model utama. Eksperimen tambahan juga dilakukan terhadap konfigurasi N-gram dan pengaruh *stemming* terhadap performa model. Hasil pengujian diuraikan secara lengkap pada sub-bab pengujian sistem di bab ini.

5. Pemeliharaan (*Maintenance*)

Tahap akhir mencakup dokumentasi sistem, pengemasan model terlatih dalam format *joblib* untuk *deployment* ke VPS, serta identifikasi potensi pengembangan lanjutan seperti perluasan dataset, peningkatan teknik normalisasi karakter, dan pengembangan mekanisme *approval queue* untuk *endpoint /report* pada implementasi produksi.

4.3 Rancangan Dataset

4.3.1 Sumber dan Metode Pengumpulan Data

Dataset yang digunakan dalam penelitian ini dikumpulkan secara langsung dari platform YouTube melalui teknik *web scraping* menggunakan *Chrome Extension* berbasis Node.js yang dikembangkan secara khusus. Data dikumpulkan dari video-video YouTube berbahasa Indonesia yang memiliki tingkat interaksi tinggi dan berpotensi menjadi target penyebaran komentar promosi judi *online*. Pengumpulan data dilakukan secara manual dengan cara memilih video yang relevan, mengaktifkan ekstensi *scraper*, lalu membiarkan sistem mengekstrak seluruh komentar secara otomatis dari elemen DOM halaman video.

Komentar hasil *scraping* disimpan dalam dua file terpisah: `final_spam.json` untuk komentar yang teridentifikasi sebagai spam, dan `final_non_spam.json` untuk

komentar yang teridentifikasi sebagai normal. Sebelum masuk ke dataset pelatihan, setiap komentar spam melewati proses penyaringan berbasis skor heuristik (*spam_score*) yang dihitung secara otomatis oleh sistem berdasarkan kehadiran sinyal-sinyal primer seperti nama *brand* judi *online* dan tautan kontak, serta sinyal sekunder seperti rasio emoji dan pola karakter Unicode. Hanya komentar dengan skor heuristik lebih dari atau sama dengan 80 yang dimasukkan ke dataset pelatihan, untuk memastikan kualitas label dan meminimalkan *noise*.

4.3.2 Proses Pelabelan Data

Setiap komentar dalam dataset diberi label biner: **spam** untuk komentar yang mengandung promosi judi *online*, dan **non-spam** (*ham*) untuk komentar normal yang tidak mengandung unsur promosi. Proses pelabelan dilakukan melalui dua mekanisme. Pelabelan otomatis dilakukan oleh sistem *scraper* berdasarkan skor heuristik. Pelabelan manual dilakukan melalui file *manual_overrides.csv* yang memuat koreksi terhadap label yang dianggap tidak akurat, termasuk pemindahan komentar dari kelas spam ke non-spam (untuk kasus *false positive*) maupun sebaliknya.

Tantangan khusus dalam pelabelan dataset ini adalah keberadaan komentar yang menyebut nama situs judi *online* dalam konteks kritik atau peringatan, bukan promosi. Komentar seperti "*tolong blokir situs XYZ*" secara semantis berbeda dari "*daftar sekarang di situs XYZ*" meskipun keduanya mengandung nama situs yang sama. Komentar jenis pertama dikategorikan sebagai non-spam karena tidak bersifat promotif.

4.3.3 Komposisi dan Pembagian Dataset

Dataset final disimpan dalam file `data/comments.csv` yang merupakan hasil penggabungan dan pengacakan (*shuffling*) seluruh data spam dan non-spam. Dataset terdiri dari 2.332 komentar berlabel spam dan 4.358 komentar berlabel non-spam, dengan total keseluruhan 6.690 baris data. Dataset kemudian dibagi menjadi dua subset menggunakan fungsi `train_test_split` dari pustaka Scikit-learn dengan rasio 80:20 — 80% untuk data latih (*training set*) dan 20% untuk data uji (*test set*), menghasilkan 5.352 data latih dan 1.338 data uji. Parameter `stratify=y` digunakan untuk memastikan proporsi kelas spam dan non-spam terjaga secara seimbang di kedua subset, sedangkan `random_state=42` digunakan untuk memastikan hasil pembagian dapat direproduksi secara konsisten.

Selain dataset utama, penelitian ini juga menyiapkan `data/hard_test_set.csv` sebagai dataset uji khusus (*hard test set*) yang berisi komentar-komentar dengan tingkat ambiguitas tinggi, yaitu komentar non-spam yang secara permukaan menyerupai spam karena mengandung nama situs atau kata kunci judi *online* dalam konteks diskusi atau kritik. Dataset ini digunakan untuk menguji ketangguhan model terhadap kasus-kasus tepi (*edge cases*) yang sulit diklasifikasikan.

Tabel 4.3 Komposisi Dataset

Subset	Jumlah Data	Keterangan
Komentar Spam	2.332	34,9% dari total dataset
Komentar Non-Spam	4.358	65,1% dari total dataset
Total Dataset	6.690	Sebelum pembagian <i>train-test</i>
Data Latih (80%)	5.352	Digunakan untuk melatih model

Data Uji (20%)	1.338	Digunakan untuk evaluasi akhir
Hard Test Set	Terpisah	Kasus tepi dengan ambiguitas tinggi

4.4 Pipeline Sistem

Pipeline sistem deteksi spam komentar judi *online* dalam penelitian ini terdiri dari dua alur utama yang berjalan secara terintegrasi: alur pelatihan model (*training pipeline*) yang berjalan secara *offline*, dan alur inferensi (*inference pipeline*) yang berjalan secara *real-time* saat ekstensi aktif di peramban pengguna.

4.4.1 Alur Pelatihan Model (*Training Pipeline*)

Alur pelatihan model merupakan proses satu kali yang menghasilkan model SVM terlatih yang siap digunakan untuk klasifikasi. Tahapan dalam alur ini adalah sebagai berikut:

1. Pengumpulan Data

Komentar YouTube dikumpulkan melalui *scraper* berbasis Node.js dan disimpan dalam format JSON terpisah untuk kelas spam dan non-spam.

2. Persiapan Dataset

Skrip `prepare_dataset.py` memfilter data berdasarkan skor heuristik, menerapkan koreksi manual, lalu menggabungkan dan mengacak seluruh data menjadi satu file CSV.

3. Prapemrosesan Teks

Seluruh teks komentar diproses melalui pipeline normalisasi di `preprocessing.py`, mulai dari penghapusan karakter *zero-width*, normalisasi Unicode NFKC, normalisasi *leet speak*, kanonisasi nama *brand judi online*, hingga penghapusan *stopword*.

4. Ekstraksi Fitur TF-IDF

Teks yang telah bersih dikonversi menjadi matriks numerik menggunakan `TfidfVectorizer` dengan parameter `ngram_range=(1,2)`, `max_features=10.000`, `sublinear_tf=True`, dan `min_df=2`.

5. Pembagian Dataset

Data dibagi menjadi 80% data latih dan 20% data uji menggunakan `train_test_split` dengan parameter `stratify=y` untuk menjaga proporsi kelas.

6. Pencarian Parameter Terbaik

`GridSearchCV` menguji lima kandidat nilai parameter `C` (0,01 / 0,1 / 1 / 10 / 100) melalui *5-fold cross-validation* untuk menemukan konfigurasi SVM yang menghasilkan *F1-macro* tertinggi.

7. Pelatihan Model

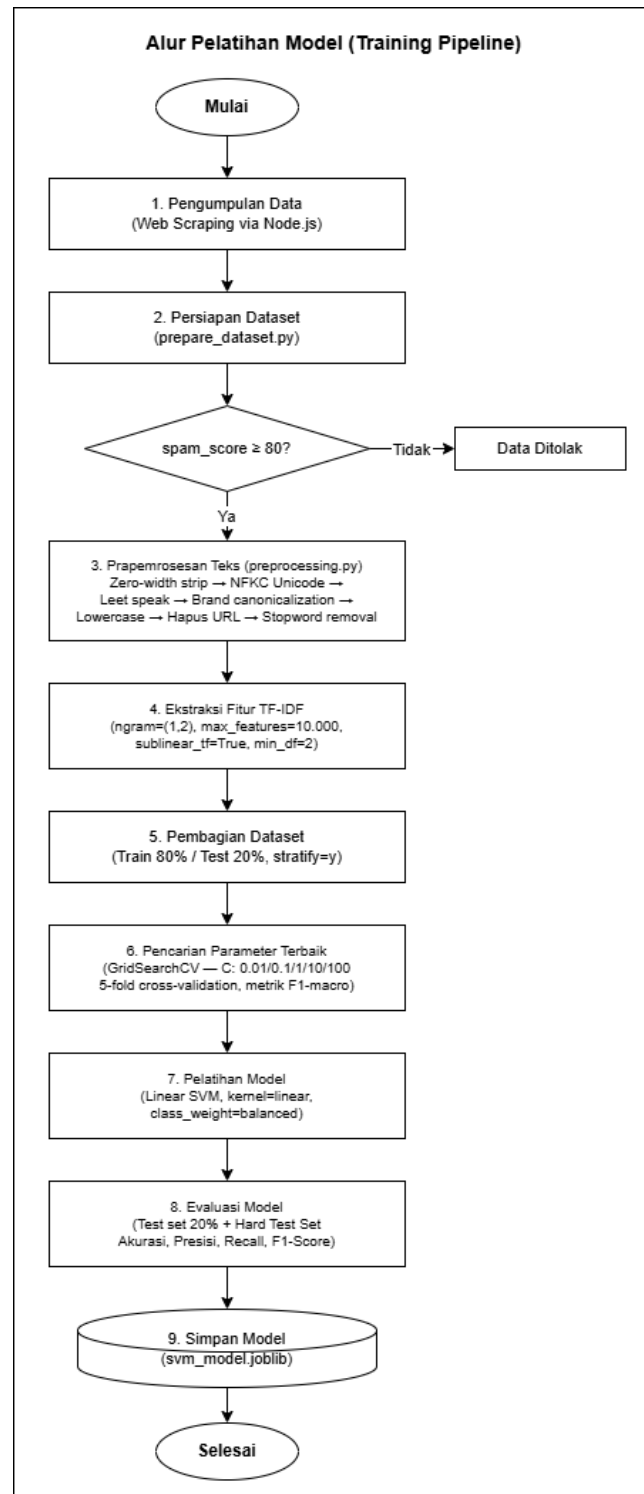
Representasi numerik digunakan untuk melatih model Linear SVM dengan parameter `kernel='linear'` dan `class_weight='balanced'` menggunakan nilai `C` terbaik dari langkah sebelumnya.

8. Evaluasi dan Penyimpanan

Model dievaluasi menggunakan data uji 20% dan *hard test set*, lalu disimpan dalam format *joblib* (`svm_model.joblib`) untuk di-*deploy* ke VPS.

Selama proses pengembangan, sistem juga menyediakan *endpoint* /report yang memungkinkan peneliti memberikan koreksi label secara langsung dari antarmuka ekstensi ketika ditemukan kesalahan klasifikasi. Data koreksi tersebut otomatis ditambahkan ke `comments.csv` dan `manual_overrides.csv`, sehingga dapat digunakan sebagai masukan tambahan pada iterasi pelatihan model berikutnya.

Mekanisme ini bersifat *development-only* dan tidak diaktifkan pada implementasi produksi.



Gambar 4.1 Alur Pelatihan Model (Training Pipeline)

4.4.2 Alur Inferensi Real-Time (*Inference Pipeline*)

Alur inferensi berjalan setiap kali pengguna membuka halaman YouTube dengan ekstensi aktif. Tahapannya adalah sebagai berikut:

1. Deteksi Komentar

Content script menggunakan *MutationObserver* untuk memantau perubahan DOM secara *asynchronous* dan menangkap teks komentar baru yang dimuat secara dinamis.

2. Pengiriman ke Server

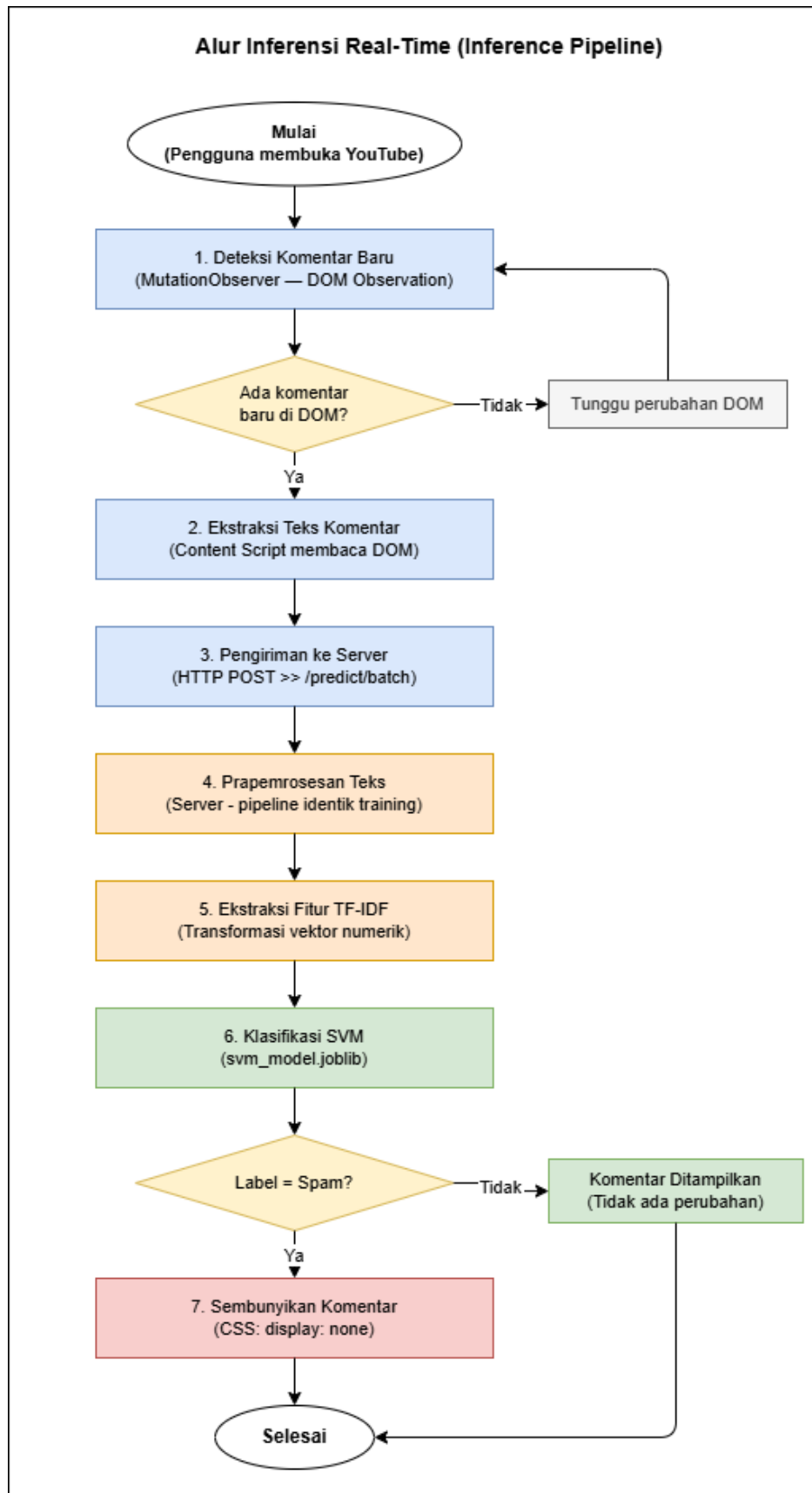
Teks komentar dikemas dalam format JSON dan dikirim ke server FastAPI yang berjalan di VPS melalui HTTP POST ke *endpoint* `/predict/batch` untuk efisiensi pemrosesan.

3. Prapemrosesan dan Klasifikasi

Server menjalankan pipeline prapemrosesan yang identik dengan tahap pelatihan, mengekstrak fitur TF-IDF, lalu mengklasifikasikan teks menggunakan model SVM yang telah dimuat dari file *joblib*.

4. Manipulasi DOM

Hasil prediksi dikembalikan ke *content script*. Komentar dengan label "spam" dan *confidence score* di atas *threshold* yang ditetapkan diproses sesuai mode yang dipilih pengguna. Pada mode **Redupkan**, komentar ditampilkan dengan transparansi 15% disertai *badge* yang menunjukkan persentase kepercayaan model dan dapat diklik untuk memperlihatkan komentar kembali. Pada mode **Hilangkan**, komentar disembunyikan sepenuhnya melalui injeksi properti *CSS display: none*.



Gambar 4.2 Alur Inferensi Real-Time (Inference Pipeline)

4.5 Perancangan Model

4.5.1 Konfigurasi Ekstraksi Fitur TF-IDF

Proses ekstraksi fitur TF-IDF pada penelitian ini menggunakan fungsi `TfidfVectorizer` dari pustaka `Scikit-learn`, yang dikonfigurasi dengan empat pengaturan utama.

Pengaturan pertama adalah `max_features=10.000`, yang berarti sistem hanya akan mempertahankan 10.000 kata atau frasa dengan bobot TF-IDF tertinggi dari keseluruhan data, dan membuang sisanya. Tujuannya supaya jumlah fitur tidak membengkak tanpa batas, mengingat data komentar bisa mengandung puluhan ribu kata unik yang berbeda-beda.

Pengaturan kedua adalah `ngram_range=(1, 2)`, yang berarti sistem tidak hanya menghitung kata secara satu per satu (*unigram*), tetapi juga menghitung pasangan dua kata yang berurutan (*bigram*). Pengaturan ini penting karena beberapa frasa dalam konteks judi *online* memiliki makna yang berbeda ketika dipisah dibandingkan ketika digabung. Sebagai contoh, kata "slot" dan kata "gacor" jika dihitung terpisah tidak terlalu mencurigakan, tetapi frasa "slot gacor" sebagai satu kesatuan jauh lebih kuat sebagai penanda komentar promosi judi.

Pengaturan ketiga adalah `min_df=2`, yang berarti sistem akan mengabaikan kata yang hanya muncul pada satu komentar saja di seluruh dataset. Kata yang hanya muncul sekali biasanya tidak cukup informatif untuk dijadikan pola umum, dan justru berisiko membuat model "menghafal" kasus tertentu alih-alih mempelajari pola yang berlaku secara luas.

Pengaturan keempat adalah `sublinear_tf=True`, yang mengubah cara sistem menghitung frekuensi kemunculan kata. Alih-alih menghitung jumlah kemunculan

kata secara apa adanya, sistem menerapkan rumus logaritma sehingga kata yang muncul berkali-kali dalam satu komentar tidak dianggap berkali-kali lebih penting. Misalnya, kata yang muncul 20 kali dalam satu komentar tidak dianggap 20 kali lebih penting dibandingkan kata yang hanya muncul sekali, melainkan hanya sedikit lebih penting saja.

4.5.2 Konfigurasi Model Linear SVM

Model klasifikasi pada penelitian ini menggunakan SVC (*Support Vector Classifier*) dari Scikit-learn dengan kernel linear (kernel='linear'), sesuai dengan alasan yang telah dijelaskan pada Sub-bab 2.6 bahwa kernel linear lebih cocok untuk data berdimensi tinggi seperti hasil ekstraksi TF-IDF.

Pengaturan class_weight='balanced' digunakan agar model tidak condong ke satu kelas saja apabila jumlah data spam dan non-spam tidak seimbang. Sebagai gambaran, jika dataset berisi lebih banyak komentar non-spam dibandingkan spam, model tanpa pengaturan ini cenderung "malas" mempelajari ciri-ciri komentar spam karena jumlahnya lebih sedikit. Pengaturan class_weight='balanced' memberi bobot lebih besar pada kelas yang jumlahnya lebih sedikit, sehingga model tetap belajar mengenali kedua kelas secara adil.

Pengaturan random_state=42 ditetapkan agar hasil pelatihan model selalu sama setiap kali dijalankan ulang, sehingga hasilnya dapat direproduksi secara konsisten.

4.5.3 Pencarian Nilai Parameter Terbaik (*GridSearchCV*)

Nilai parameter regularisasi C pada SVM menentukan keseimbangan antara margin yang lebar dan toleransi terhadap kesalahan klasifikasi pada data latih. Nilai C yang kecil menghasilkan margin yang lebih lebar namun berisiko *underfitting*, sedangkan nilai C yang besar menghasilkan batas keputusan yang lebih ketat namun

berisiko *overfitting*. Untuk menentukan nilai C yang optimal secara objektif, penelitian ini menggunakan GridSearchCV yang menguji lima kandidat nilai C — yaitu 0,01, 0,1, 1, 10, dan 100 — melalui *5-fold cross-validation* pada data latih dengan metrik evaluasi *F1-macro*. Nilai C yang menghasilkan skor *F1-macro* rata-rata tertinggi pada seluruh *fold* dipilih sebagai parameter final model.

4.5.4 Validasi Model (*5-Fold Cross-Validation*)

Selain evaluasi menggunakan pembagian data 80:20, penelitian ini juga melakukan *5-fold cross-validation* terhadap keseluruhan dataset. Pada setiap *fold*, model dilatih ulang menggunakan pipeline yang identik dan dievaluasi menggunakan metrik *F1-macro*. Cara kerjanya, seluruh data dibagi menjadi 5 bagian yang kurang lebih sama besar, lalu model dilatih dan diuji sebanyak 5 kali secara bergantian — setiap kali menggunakan satu bagian sebagai data uji dan empat bagian lainnya sebagai data latih. Skor rata-rata dan standar deviasi dari kelima *fold* tersebut dilaporkan untuk menunjukkan tingkat konsistensi performa model pada subset data yang berbeda-beda.

4.5.5 Pertimbangan *Stemming*

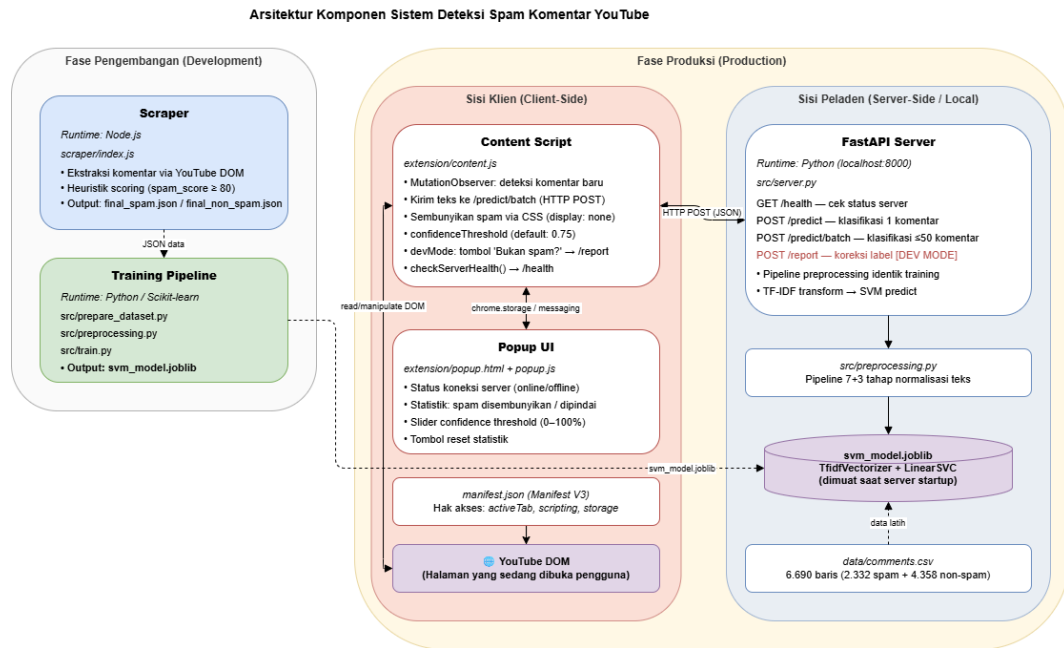
Sebagai bagian dari proses eksplorasi metodologi, dilakukan eksperimen terpisah untuk menguji pengaruh *stemming* Sastrawi terhadap performa model. Hasil eksperimen menunjukkan peningkatan *F1-macro* yang tidak signifikan sebesar +0,0002, sehingga *stemming* diputuskan tidak diintegrasikan ke dalam pipeline prapemrosesan produksi. Detail hasil eksperimen ini dibahas lebih lanjut pada sub-bab pengujian sistem di bab ini.

4.5.6 Pertimbangan Pendekatan Hibrida

Pada tahap perancangan awal, sempat dipertimbangkan pendekatan hibrida yang menggabungkan prediksi model SVM dengan aturan berbasis kata kunci (*rule-based*) sebagai lapisan tambahan, dengan tujuan menutup potensi kelemahan representasi *bag-of-words* dalam mengenali pola spam yang sangat eksplisit. Pendekatan ini diimplementasikan sebagai fitur yang dapat diaktifkan atau dinonaktifkan (ENABLE_HYBRID_RULES) untuk memungkinkan pengujian perbandingan secara terkontrol. Hasil pengujian dan keputusan akhir mengenai pendekatan ini dibahas pada sub-bab pengujian sistem di bab ini.

4.6 Arsitektur Komponen Sistem

Sistem yang dikembangkan dalam penelitian ini terdiri dari tiga komponen utama yang saling terintegrasi: *scraper* berbasis Node.js untuk pengumpulan data, ekstensi Chrome berbasis Manifest V3 sebagai antarmuka pengguna di sisi klien, dan server FastAPI berbasis Python sebagai mesin pemrosesan di sisi peladen. Ketiga komponen ini beroperasi secara independen namun terhubung melalui protokol HTTP.



Gambar 4.3 Diagram Arsitektur Komponen Sistem

4.6.1 Komponen *Scraper* (Node.js)

Komponen *scraper* merupakan skrip berbasis Node.js yang berjalan secara mandiri di lingkungan terminal dan bertugas mengumpulkan data komentar YouTube untuk keperluan pelatihan model. Komponen ini tidak terlibat dalam alur operasional sistem saat ekstensi digunakan oleh pengguna akhir — perannya murni sebagai alat pengumpulan data pada fase pengembangan. File utama komponen ini adalah `scraper/index.js` yang menerima ID video YouTube sebagai argumen, mengekstrak seluruh komentar beserta metadata-nya, lalu menyimpan hasilnya dalam format JSON ke direktori data.

4.6.2 Komponen *Chrome Extension* (Manifest V3)

Komponen ekstensi Chrome merupakan antarmuka utama sistem yang berinteraksi langsung dengan pengguna dan halaman YouTube. Komponen ini terdiri dari tiga file inti yang memiliki peran berbeda.

Content script (extension/content.js) adalah skrip yang diinjeksi langsung ke halaman YouTube dan bertanggung jawab atas seluruh logika deteksi secara *real-time*. Skrip ini menggunakan MutationObserver untuk memantau perubahan DOM, mengekstrak teks komentar baru, mengirimkannya ke server melalui *endpoint* /predict/batch, lalu menyembunyikan komentar yang diklasifikasikan sebagai spam melalui manipulasi CSS. Skrip ini juga mengelola dua parameter operasional: *confidenceThreshold* yang menentukan seberapa yakin model harus sebelum sebuah komentar disembunyikan dengan nilai *default* 0,75, dan *devMode* yang mengaktifkan tampilan visual tambahan berupa *badge* merah dan transparansi komentar untuk keperluan koreksi dataset selama pengembangan.

Popup (extension/popup.html dan extension/popup.js) adalah antarmuka grafis yang muncul ketika pengguna mengklik ikon ekstensi di peramban. Antarmuka ini menampilkan status koneksi ke server, statistik jumlah komentar yang telah dipindai dan disembunyikan, serta kontrol untuk mengatur nilai *confidence threshold* melalui *slider* yang dapat digeser secara langsung oleh pengguna.

4.6.3 Komponen Server FastAPI (Python)

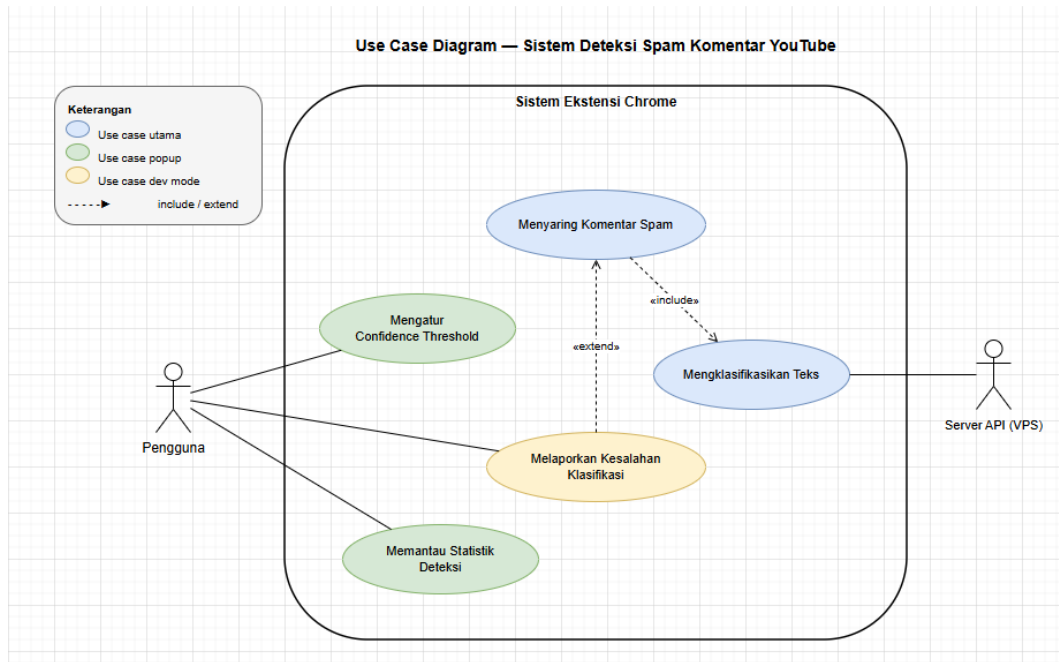
Komponen server merupakan mesin pemrosesan kecerdasan buatan yang di-*deploy* pada *Virtual Private Server* (VPS) berbasis Ubuntu Server 24.04 LTS dan dapat diakses melalui jaringan internet. Server ini memuat model SVM terlatih dari file `svm_model.joblib` pada saat pertama kali dijalankan, dan menyediakan empat *endpoint* REST API yang dapat diakses oleh ekstensi:

- /health : Memeriksa status ketersediaan server dan memastikan model telah dimuat dengan benar.

- `/predict` : Menerima satu teks komentar dan mengembalikan label klasifikasi beserta skor kepercayaan (*confidence score*).
- `/predict/batch` : Menerima hingga 50 teks komentar dalam satu permintaan dan mengembalikan hasil klasifikasi untuk seluruh komentar dalam satu respons, sehingga lebih efisien dibandingkan mengirim permintaan satu per satu.
- `Report` : Menerima koreksi label dari pengguna dan menyimpannya ke dataset untuk keperluan pelatihan ulang di masa mendatang. *Endpoint* ini aktif pada mode pengembangan (*development mode*) dan pada implementasi produksi memerlukan mekanisme antrian persetujuan (*approval queue*) untuk mencegah penyalahgunaan.

4.7 Diagram Use Case

Use case diagram digunakan untuk menggambarkan interaksi fungsional antara aktor dengan sistem dari perspektif pengguna. Diagram ini menunjukkan batasan sistem (*system boundary*), aktor yang terlibat, serta fungsi-fungsi utama yang dapat dijalankan oleh sistem.



Gambar 4.4 Use Case Diagram Sistem Deteksi Spam Komentar YouTube

Berdasarkan *use case diagram* di atas, sistem melibatkan dua aktor dan lima *use case* utama yang dijabarkan sebagai berikut:

Aktor:

1. Pengguna

Pengguna akhir yang membuka halaman YouTube menggunakan peramban Chrome dengan ekstensi aktif. Pengguna berinteraksi dengan sistem melalui antarmuka *popup* untuk mengatur *confidence threshold*, memantau statistik deteksi, dan melaporkan kesalahan klasifikasi.

2. Server API (VPS)

Sistem eksternal yang menerima teks komentar dari ekstensi, menjalankan proses klasifikasi, dan mengembalikan hasil prediksi.

Use Cases:

1. Menyaring Komentar Spam

Use case utama yang diinisiasi ketika pengguna membuka atau menggulir halaman video YouTube. Sistem kemudian secara otomatis mendeteksi komentar baru, mengirimkannya ke server untuk diklasifikasikan, dan menyembunyikan komentar yang terdeteksi sebagai spam.

2. Mengklasifikasikan Teks

Use case yang memiliki relasi *include* dengan "Menyaring Komentar Spam", artinya setiap kali penyaringan dilakukan, klasifikasi selalu dipanggil sebagai bagian yang tidak terpisahkan. *Use case* ini dijalankan oleh Server API yang menerima teks, menjalankan pipeline prapemrosesan dan ekstraksi fitur TF-IDF, lalu menghasilkan label klasifikasi beserta *confidence score*.

3. Mengatur *Confidence Threshold*

Use case yang dijalankan pengguna melalui *popup* ekstensi untuk menyesuaikan ambang batas kepercayaan model. Nilai *threshold* yang lebih tinggi membuat sistem lebih selektif dalam menyembunyikan komentar.

4. Memantau Statistik Deteksi

Use case yang memungkinkan pengguna melihat jumlah komentar yang telah dipindai dan disembunyikan sejak ekstensi diaktifkan, ditampilkan melalui antarmuka *popup*.

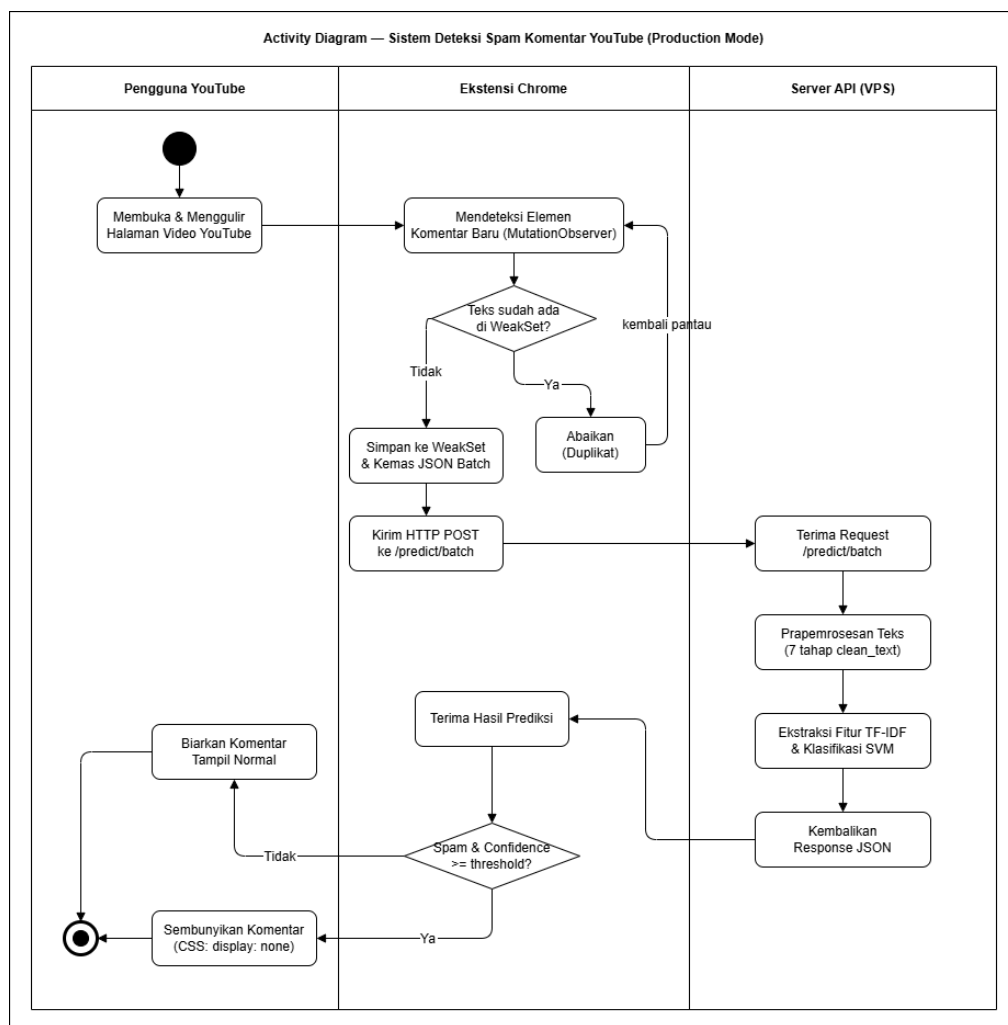
5. Melaporkan Kesalahan Klasifikasi

Use case yang memiliki relasi *extend* terhadap "Menyaring Komentar Spam", artinya *use case* ini hanya muncul secara kondisional ketika sistem berjalan dalam mode pengembangan (*dev mode*). Pengguna dapat memberikan koreksi

ketika komentar normal disembunyikan secara keliru (*false positive*) melalui tombol "Bukan spam?" yang muncul pada komentar yang disembunyikan.

4.8 Activity Diagram

Activity diagram digunakan untuk menggambarkan alur kerja sistem secara kronologis dari tahap inisiasi hingga eksekusi manipulasi antarmuka. Diagram ini disajikan dalam format swimlane yang membagi tanggung jawab proses ke dalam tiga jalur: Pengguna YouTube, Ekstensi Chrome, dan Server API.



Gambar 4.5 Activity Diagram Sistem Deteksi Spam Komentar YouTube

Berdasarkan *activity diagram* di atas, alur kerja sistem berjalan melalui tahapan berikut:

1. Inisiasi

Pengguna membuka dan menggulir halaman video YouTube, yang memicu ekstensi Chrome untuk mulai memantau perubahan DOM.

2. Deteksi Komentar

Ekstensi mendeteksi elemen komentar baru yang dimuat secara dinamis oleh YouTube. Sistem memeriksa apakah teks komentar tersebut sudah pernah diproses sebelumnya menggunakan objek WeakSet untuk mencegah pemrosesan ganda.

3. Pengiriman ke Server

Teks komentar yang unik dikemas dalam format JSON dan dikirim ke Server API melalui HTTP POST ke *endpoint* /predict/batch.

4. Pemrosesan di Server

Server API menerima permintaan, menjalankan pipeline prapemrosesan teks yang identik dengan tahap pelatihan, mengekstrak fitur TF-IDF, lalu mengklasifikasikan teks menggunakan model SVM. Server mengembalikan respons JSON yang memuat label klasifikasi dan *confidence score* untuk setiap komentar.

5. Evaluasi Hasil

Ekstensi menerima respons dan mengevaluasi setiap hasil klasifikasi. Komentar dengan label spam dan *confidence score* di atas *threshold* yang ditetapkan (nilai *default* 0,75) diteruskan ke tahap manipulasi DOM.

6. Manipulasi DOM

Komentar yang memenuhi syarat spam disembunyikan dari tampilan pengguna melalui injeksi properti CSS `display: none`. Komentar yang tidak memenuhi syarat dibiarkan tampil normal.

Perlu dicatat bahwa diagram di atas menggambarkan alur sistem dalam mode produksi (*production mode*). Pada mode pengembangan (*dev mode*), komentar yang terdeteksi sebagai spam tidak langsung disembunyikan sepenuhnya melainkan ditampilkan dengan transparansi 15% disertai *badge* merah yang menunjukkan persentase *confidence score*, serta tombol "Bukan spam?" untuk keperluan koreksi dataset. Perbedaan tampilan antara kedua mode ini diuraikan lebih lanjut pada sub-bab Rancangan Tampilan.

4.9 Sequence Diagram

4.10 Rancangan Tampilan

4.11 Pengujian Sistem

4.12 Analisis Kelayakan Sistem

DAFTAR PUSTAKA

- Abdillah, W. F., Premana, A., & Bhakti, R. M. H. (2021). Analisis Sentimen Penanganan COVID-19 dengan Support Vector Machine: Evaluasi Leksikon dan Metode Ekstraksi Fitur. *Jurnal Ilmiah Intech: Information Technology Journal of UMUS*, 3(2).
- Airlangga, G. (2024a). Spam detection on YouTube comments using advanced machine learning models: A comparative study. *Brilliance: Research of Artificial Intelligence*, 4(2), 500–508. <https://doi.org/10.47709/brilliance.v4i2.4670>
- Airlangga, G. (2024b). Spam detection in YouTube comments using deep learning models: A comparative study of MLP, CNN, LSTM, BiLSTM, GRU, and attention mechanisms. *MALCOM: Indonesian Journal of Machine Learning and Computer Science*, 4(4), 1533–1538. <https://doi.org/10.57152/malcom.v4i4.1671>
- Angelo, M., Robet, R., & Hendrik, J. (2025). Comparative Performance of Machine Learning Algorithms for Detecting Online Gambling Promotional Comments on YouTube. *Jurnal Teknologi dan Manajemen Informatika*, 11(2), 132–142. <https://doi.org/10.26905/jtmi.v11i2.16286>
- Anis, Y., Wahyudi, E. N., & Kurniawan, H. C. (2024). Metode Waterfall dalam pengembangan sistem inventaris guna meningkatkan efisiensi manajemen stok barang. *Jurnal Teknologi dan Sistem Informasi Bisnis*, 6(1). <https://doi.org/10.47233/jteksis.v6i1.1351>
- Apricia, P., Wulandari, H. R., Agustian, F., Ananda, F. A. N., & Pratama, J. A. (2024). Penerapan metode Naive Bayes dalam klasifikasi spam SMS menggunakan fitur teks untuk mengatasi ancaman pada pengguna. *Journal of Information System Research*, 5(3), 873–880. <https://ejurnal.seminar-id.com/index.php/josh/article/view/5070>
- Ardiansyah, Nasution, A. M., & Novelan, M. S. (2025). Implementasi kecerdasan buatan dalam deteksi cybercrime: Komparasi model Naive Bayes dan SVM pada pola komentar judi online. *Indonesian Journal of Education and Computer Science*, 3(3), 170–179. <https://jurnal.intekom.id/index.php/indotech/article/view/1762>
- Azhari, R. Y. (2022). Web service framework: Flask dan FastAPI. *Technology and Informatics Insight Journal*, 1(1), 80–87. <https://doi.org/10.32639/tij.v1i1.54>
- Chua, S., Tan, A., Nohuddin, P. N. E., & Ahmad Hijazi, M. H. (2024). Comparing the Effectiveness and Efficiency of Machine Learning Models for Spam Detection on Twitter. *Journal of Advanced Research in Applied Sciences and Engineering Technology*, 60(2), 153–164.

Helmiyah, S., & Pramestiawan, R. (2025). Analisis komparatif algoritma machine learning dengan metrik akurasi, presisi, recall, dan F1-score pada dataset kacang kering. *Jurnal Ilmu Komputer dan Teknologi*, 6(3), 152–159. <https://doi.org/10.35960/ikomti.v6i3.2031>

Herawati, D. A., Risdhianto, A., & Saptono, E. (2025). Indonesia darurat judi online: Judi online sebagai ancaman nirmiliter terhadap ketahanan nasional. *CITIZEN: Jurnal Ilmiah Multidisiplin Indonesia*, 5(5), 1251–1261. <https://doi.org/10.53866/jimi.v5i5.991>

Iriananda, S. W., Budiawan, R. W., Rahman, A. Y., & Istiadi. (2024). Optimasi klasifikasi sentimen komentar pengguna game bergerak menggunakan SVM, grid search dan kombinasi N-gram. *Jurnal Teknologi Informasi dan Ilmu Komputer*, 11(4), 835–846. <https://doi.org/10.25126/jtiik.1148244>

Kaddoura, S., Chandrasekaran, G., Popescu, D. E., & Duraisamy, J. H. (2022). A systematic literature review on spam content detection and classification. *PeerJ Computer Science*, 8, e830. <https://doi.org/10.7717/peerj-cs.830>

Khairunnisa, S., Adiwijaya, A., & Al Faraby, S. (2021). Pengaruh text preprocessing terhadap analisis sentimen komentar masyarakat pada media sosial Twitter (studi kasus pandemi COVID-19). *Jurnal Media Informatika Budidarma*, 5(2), 406–414. <https://doi.org/10.30865/mib.v5i2.2835>

Khan, R. A. (2018). Spammer detection: A study of spam filter comments on YouTube videos. *LGU International Journal for Electronic Crime Investigation*, 2(2).

Kinanti, N., & Indriyanti, A. D. (2021). PIECES framework sebagai evaluasi tingkat kepuasan terhadap penggunaan sistem informasi. *Journal of Emerging Information Systems and Business Intelligence*, 2(2), 1–7. <https://ejournal.unesa.ac.id/index.php/JEISBI/article/view/39730>

Koprawi, M., & Putra, W. S. (2023). Implementasi web scraping pada Google Cendekia sebagai sarana profiling penelitian dosen. *Science Tech: Jurnal Ilmu Pengetahuan dan Teknologi*, 9(1), 59–72. <https://doi.org/10.30738/st.vol9.no1.a14188>

Maulana, I. R. (2025). Optimasi klasifikasi data teks menggunakan algoritma logistic regression dengan TF-IDF dan SMOTE. *JIKOMTI: Jurnal Ilmiah Ilmu Komputer dan Teknologi Informasi*, 3(1). <https://ojs.sains.ac.id/index.php/Jikomti/article/view/97>

Nanda, R., Haerani, E., Gusti, S. K., & Ramadhani, S. (2022). Klasifikasi Berita Menggunakan Metode Support Vector Machine. *Jurnal Nasional Komputasi dan Teknologi Informasi*, 5(2), 269–278.

Oktavia, A. N. (2024). Implementasi metode Natural Language Processing dalam studi analisis semantik dan emosi buzzer pada tweet di aplikasi X. *BIIKMA: Buletin Ilmiah Ilmu Komputer dan Multimedia*, 2(1), 154–159.

Pratama, R., et al. (2025). Sistem Rekomendasi Wisata Kuliner di Gunungkidul Menggunakan Metode Content-Based Filtering (Penerapan Web Scraping via Ekstensi Chrome). *Jurnal Informatika dan Teknik Elektro Terapan*.

Pratama, R. Y. (2026). Pengembangan sistem chatbot cerdas berbasis Natural Language Processing (NLP) untuk peningkatan layanan informasi hotel. *Jurnal Informatika dan Teknik Elektro Terapan*, 14(1).

Ramadhan, R. F., & Fauzan, A. (2023). Pembatasan internet berbasis ekstensi web pada Chrome browser. *Prosiding Seminar Nasional Teknik (SENATEK) 2023*, 6. <https://doi.org/10.30595/pspfs.v6i.869>

Rosa, A. S., & Shalahuddin, M. (2019). *Rekayasa perangkat lunak: Terstruktur dan berorientasi objek* (Edisi revisi). Informatika.

Sari, M., & Asmendri. (2020). Penelitian Kepustakaan (Library Research) dalam Penelitian Pendidikan IPA. *Natural Science: Jurnal Penelitian Bidang IPA dan Pendidikan IPA*, 6(1), 41–53.

Sinaga, A., & Nainggolan, S. P. (2023). Analisis perbandingan akurasi dan waktu proses algoritma stemming Arifin-Setiono dan Nazief-Adriani pada dokumen teks bahasa Indonesia. *Sebatik*, 27(1), 63–69. <https://doi.org/10.46984/sebatik.v27i1.2072>

Somantri, O., Wiyono, S., & Dairoh, D. (2020). Optimalisasi Support Vektor Machine (SVM) untuk klasifikasi tema tugas akhir berbasis K-Means. *Jurnal Edukasi dan Penelitian Informatika (JEPIN)*, 6(3), 312–319.

Sugiyono. (2022). *Metode Penelitian Kuantitatif, Kualitatif, dan R&D* (Cetakan ke-4). Bandung: Alfabeta.

LAMPIRAN

Kegiatan	M1-2	M3-4	M5-6	M7-9	M10-12	M13-15
	<i>Apr-Mei</i>	<i>Mei</i>	<i>Mei-Jun</i>	<i>Jun-Jul</i>	<i>Jul</i>	<i>Jul-Agt</i>
Finalisasi proposal & dataset scraping						
Labeling & preprocessing data						
Training model SVM + evaluasi						
Bangun Chrome Extension + integrasi API						
Penulisan skripsi penuh (Bab I–V)						
Bimbingan & revisi						
Pengujian Sistem & Perbaikan Bug						
Persiapan sidang & PPT						